



"Reducing train delays in a real-time context : a constraint programming approach using conditional time-interval variables"

Thiry, Jérôme ; Felten, Florian

Abstract

Railway traffic scheduling is a problem that affects a lot of people in their everyday life. While designing efficient train schedules is already a challenge in itself, it is already well solved. However, small perturbations in the traffic can have a big impact on the initial timetable and cause disproportionate delays. Rescheduling and rerouting trains in these situations help limiting the impact of the initial perturbations. Constraint programming is an efficient programming paradigm when it comes to scheduling problems. In this thesis, the focus is placed on rescheduling using constraint programming models built with conditional time-interval variables. Firstly, an implementation of these variables is given for OscanR, which yields promising results. Secondly, some additions are proposed to existing models that target this problem. Finally, the performances of the new model are assessed on fictive data using CP Optimizer and compared to classical approaches. The tests have been rea...

Document type : *Mémoire (Thesis)*

Référence bibliographique

Thiry, Jérôme ; Felten, Florian. *Reducing train delays in a real-time context : a constraint programming approach using conditional time-interval variables*. Ecole polytechnique de Louvain, Université catholique de Louvain, 2018. Prom. : Schaus, Pierre ; Cappart, Quentin.

Reducing train delays in a real-time context

A constraint programming approach using conditional time-interval variables

Dissertation presented by
Florian FELTEN , Jérôme THIRY

for obtaining the Master's degree in
Computer Science

Supervisor(s)
Quentin CAPPART, Pierre SCHAUS

Reader(s)
Christophe LIMBRÉE, Sascha VAN CAUWELAERT

Academic year 2017–2018

Abstract

Railway traffic scheduling is a problem that affects a lot of people in their everyday life. While designing efficient train schedules is already a challenge in itself, it is already well solved. However, small perturbations in the traffic can have a big impact on the initial timetable and cause disproportionate delays.

Rescheduling and rerouting trains in these situations help limiting the impact of the initial perturbations. Constraint programming is an efficient programming paradigm when it comes to scheduling problems. In this thesis, the focus is placed on rescheduling using constraint programming models built with conditional time-interval variables. Firstly, an implementation of these variables is given for *Oscar*, which yields promising results. Secondly, some additions are proposed to existing models that target this problem. Finally, the performances of the new model are assessed on fictive data using *CP Optimizer* and compared to classical approaches. The tests have been realised on multiple Belgian stations and with scenarios of different levels of complexity. The results show that the model gives better performances than greedy approaches (generally used by the railway operators) in almost all the situations.

Acknowledgements

We would like to first thank our supervisor Quentin Cappart, who provided us with useful insights and advice, and gave us his support and time throughout the year, even when being on the other side of the Atlantic Ocean.

Then, we would like to thank our supervisor Pierre Schaus, for putting us in the right direction when we needed it the most.

We would also like to acknowledge Christophe Limbrée for his valuable comments and help on all the railway aspects.

Florian I would like to thank all the members of my family for their support through these five years, with a special mention to my mother, without who I would not be the person I am today. Last but not least, I thank all my friends who often remind me that taking a break is sometimes more productive than being locked inside in front of a computer.

Jérôme I would like to personally thank my family which provided unconditional support, my friends for well-needed moments of relaxation, and Emilie for her support and her ability to put up with my bad mood and cheer me up when things were not going the way they were supposed to.

Contents

Abstract	i
Acknowledgements	iii
1 Introduction	1
2 State of the art	3
3 Constraint programming	5
3.1 Principle	5
3.1.1 Modeling	5
3.1.2 Search	5
3.2 Constraint optimization problem	6
4 Conditional time-interval variables	7
4.1 General principle	7
4.2 Constraints	7
4.2.1 Logical constraints	8
4.2.2 Temporal constraints	8
4.2.3 Hybrid constraints	10
4.3 Propagation	10
5 CTI variables for OsaR	13
5.1 Anatomy of a conditional time-interval	13
5.2 Logical network	15
5.3 Temporal network	20
5.4 Constraints	23
5.5 Search	27
5.5.1 Branching schemes	27
5.5.2 Variable ordering heuristics	28
5.5.3 Value ordering heuristics	28
5.6 Performance	29
5.6.1 Search methods	29
5.6.2 CTI against reified constraints	30
5.7 A first conclusion and suggestions for further work	32

6	The initial model	33
6.1	Signaling principles	33
6.2	Elements	34
6.2.1	Activities	34
6.2.2	Parameters	35
6.2.3	Decision variables	36
6.3	Constraints	37
6.3.1	Precedence	37
6.3.2	Execution consistency	38
6.3.3	Alternative	38
6.3.4	Span	38
6.3.5	noOverlap - unary resource	38
6.3.6	Train Ordering consistency	39
6.4	Objective functions	39
6.4.1	Homogeneous model	39
6.4.2	Heterogeneous model	40
6.5	Experiments and results	40
6.6	Limitations of the initial model	41
6.6.1	Length of the trains	41
6.6.2	Platforms	41
6.6.3	Considering larger areas and multiple stations	41
6.6.4	Encountering signals after route activation	41
7	The new model	43
7.1	Length of the trains	43
7.1.1	The problem	43
7.1.2	Parts of the model to be modified	44
7.1.3	Solutions	45
7.2	Platforms	54
7.2.1	The problem	54
7.2.2	Parts of the model to be modified	54
7.2.3	Solutions	55
7.3	Larger areas and multiple stations	56
7.3.1	The problem	56
7.3.2	Parts of the model to be modified	56
7.3.3	Solution	56
7.4	Encountering signals after route activation	58
7.4.1	Reservation of the whole route after activation	58
7.4.2	Reservation of the elements until a signal or a platform	58
7.5	Conclusion about the new model	59
7.5.1	Advantages	59
7.5.2	Drawbacks	60
7.6	Further work	60

7.6.1	Considering the type of platforms	60
7.6.2	Variable speed of the trains	60
7.6.3	Considering train shunting	60
7.6.4	Introduction of disturbances	61
8	Experiments	63
8.1	Homogeneous traffic	64
8.2	Heterogeneous traffic	66
8.3	Critical situations	67
8.4	Scability	68
8.5	Reproducibility	68
9	Conclusion	69
A	CPCTIVar API	73
B	Ottignies and Wavre	79
C	Performances comparison: CP Optimizer and OsaR	81
D	OsaR models for performance assessment	83

List of symbols and abbreviations

CO Conflict Ordering	HPFS Highest Priority First-Served
COP Constraint Optimization Problem	LC Last Conflict
CP Constraint Programming	RER <i>Réseau Express Régional</i>
CSP Constraint Satisfaction Problem	OscAR Operational Research in Scala
CTI Conditional Time-Interval	SCC Strongly Connected Components
DFS Depth-First Search	TMS Traffic Management System
FCFS First Come First-Served	TTR Timetable Rescheduling
HDFS Highest Delay First-Served	

List of Figures

3.1	Examples of branching decisions	6
4.1	Overlap between activities a and b	10
4.2	Representation of type of activities	11
4.3	Representation of links between activities	11
4.4	Initial state of the activities	11
4.5	State of the activities after propagation	12
5.1	Composition of a CTI variable	14
5.2	Representation of the link $x(i) \rightarrow \neg x(j)$ in the logical network	16
5.3	The initial direct successors of A are B, C and D	18
5.4	After the transitive closure, the direct successors of A are B, C, D, E, F and G	18
5.5	Summary of the two procedures in the logical network	19
5.6	Representation of the link $s(i) \xrightarrow{6} e(j)$ in the temporal network	21
5.7	Propagation triggers on the networks	23
5.8	Propagation triggers on the constraints	25
5.9	Mean solving time per search method, models 1 and 2	30
5.10	Mean solving time per search method, models 3 and 4	30
5.11	Paired comparison of mean solving times for models 1 and 2	31
5.12	Paired comparison of mean solving times for models 3 and 4	31
6.1	Layout of a fictive train station	34
6.2	Breakdown structure of the alternative and span constraints on activities [6]	35
6.3	Position of the train on a station	36
6.4	Two trains waiting on the same track segment [6]	39
7.1	Possible positions of a train on a track segment when considering its length	43
7.2	Possible positions of a train in front of a signal when considering its length	44
7.3	Train waiting at signal having its rear carriages occupying preceding track segments	44
7.4	Figure 7.3 updated with sizeLeft and the $E_{s-e}^{t,it,r}$ set	47
7.5	Part of the breakdown structure when adding the end synchronized tail activities to the model	49
7.6	Part of the breakdown structure when adding the start synchronized tail activities to the model	53
7.7	Example of a toy station with a platform	55
7.8	Example of a toy station with consecutive itineraries	57

7.9	Example of a route split into multiple subroutes	58
8.1	Evolution of the computing time needed to obtain a first solution over 100 random homogeneous instances with an horizon of 30 minutes	68
B.1	The train stations of Wavre and Ottignies	80

List of Tables

5.1	Comparison between possible structures for the temporal network	15
5.2	Binary temporal constraints	24
5.3	Search methods and their abbreviations	29
5.4	Mean number of nodes per model and per number of variables	32
6.1	Parameters of trains and track segments in the model	36
7.1	Correspondence of elements and resources per activity with end synchronized tail activities	50
7.2	Correspondence of elements and resources per activity with end synchronized tail activities	53
7.3	Parameters of trains and track segments after the addition of train length, platforms and consecutive itineraries	57
8.1	Results of CP and classical approaches for an homogeneous traffic and 3 minutes of decision time: delays, number of improvements and optimal solutions	65
8.2	Results of CP and classical approaches for an homogeneous traffic and 3 minutes of decision time: improvement ratios	65
8.3	Results of CP and classical approach (HPFS) for an heterogeneous traffic and 3 minutes of decision time	66
8.4	Comparison between a critical CP model (10 seconds of decision time) and a normal CP model (3 minutes of decision time) on an homogeneous traffic	67
C.1	Comparison of the performances of CP Optimizer and OsaR (homogeneous model)	81
C.2	Comparison of the performances of CP Optimizer and OsaR (heterogeneous model)	81

Chapter 1

Introduction

Since its introduction in Belgium during the 19th century, the railway system has faced tremendous changes, including an ever growing traffic demand. To match this growth, the railway companies have no other choice than to develop their offer by increasing the number of trains on their infrastructure. This brings that infrastructure close to its full capacity, which amplifies the consequences of traffic perturbations. To cope with these, the obvious solution is to expand the infrastructure. However, in addition to the large financial costs it involves, it is difficult to implement this expansion in high population density areas, such as in Belgium. A good example of the difficulty of building new infrastructures can be given by the RER project, whose deadline keeps on being reported.

The other way of solving this issue is to optimize the fluidity of the growing traffic on the existing infrastructure. This is usually done by designing realistic timetables. The current solutions already give satisfying performances according to Cappart [5]. However, problems start to arise when perturbations (*e.g.* delays or technical failures) occur. Usually, in such cases, the decision of rerouting or rescheduling trains is made manually in real time by a railway operator using the available information and its expertise. [5] identifies several difficulties which may complicate these decisions:

- The time available to analyze the situation and make a decision is sometimes short (less than one minute).
- The effects of the actions that are taken are sometimes difficult to predict, especially on large stations with dense traffic.
- There are various parameters to take into account (type of trains, number of passengers, etc.) when making the decisions.

In this thesis, we focus on extending a Constraint Programming model introduced by Cappart and Schaus [6], which aims at helping the operator to make the decision having the least impact on the traffic. This model is based on conditional time-interval variables introduced by Laborie et al. [14,15].

This thesis is divided as follows: Chapter 2 presents the state of the art when it comes to rescheduling. Chapters 3, 4 and 5 constitute the first part of the thesis, which focuses on constraint programming and conditional time-intervals (CTI), and Chapters 6, 7 and 8 constitute the second part of the thesis, focusing on rescheduling train traffic using these CTI variables.

More precisely, Chapter 3 promptly presents the Constraint Programming paradigm, and how it can be used for optimization.

Chapter 4 introduces the concept of CTI variables for constraint programming in a generic way.

Chapter 5 proposes an implementation of these CTI variables for OscaR [18]. The performances of some models using these variables are then assessed using various search techniques, and the models are compared to their CTI-less counterparts. Possible further work is also explored in order to reach state-of-the-art performances.

Chapter 6 marks the beginning of the second part, focusing on rescheduling. This chapter introduces the model of Cappart and Schaus [6] and identifies some of its limitations.

Chapter 7 presents the additions to the model based on the limitations identified in the previous chapter. It is concluded with the advantages and drawbacks of the newly introduced model as well as some recommendations of further work to be conducted.

Chapter 8 assesses the performances of the new model with CP Optimizer [13], based on multiple experiments.

Chapter 2

State of the art

This chapter discusses the research and state of the art methods used to solve the optimization of rescheduling of trains in real time.

There is a huge interest going on around the studied topic, as it has been for the last 20 years. Thus, a considerable amount of papers covering the problem of timetable rescheduling (TTR) have been published recently.

Cacchiani et al. [4] give a nice overview of those. This paper covers both cases of disturbances (small perturbations that needs only a change in the timetable to be solved) and disruption (bigger perturbations that need changes in the rolling stocks and/or the staff as well) management. It also divides the research in two parts, depending on the level of details considered: the macroscopic view considers the train stations as nodes of a graph and tracks as edges without considering the elements which compose the stations (signals, track sections, etc.) while the microscopic view includes these parts of the stations. A distinction between optimization of train delays and optimization of passengers' satisfaction is also made in the paper.

The model presented in this document deals with disturbances at a microscopic level and both optimization of train delays and passenger's satisfaction are treated. Lots of related research use the *Alternative Graph* model as presented in [17] to solve the no-store job-shop scheduling problem. In fact, the problem of rescheduling the trains has been shown to be reducible to the latter, which is NP-Complete. D'Ariano et al. [9] propose a branch-and-bound algorithm with dynamic and static implication rules to schedule trains in real-time which has been integrated to the traffic management system ROMA [8]. This algorithm has been improved in multiple papers, for example [10] proposes enhancements with flexible timetables.

Usually, a *Mixed Integer Programming* (MIP) model is used to solve TTR problems. It is an optimization technique where the decision variables are either continuous (for instance to represent the time at which a train enters a station) or discrete (for example to represent the fact that a train will use a route or not). However, "*the efficiency of such models to solve scheduling problems is shown to be highly dependable on the chosen granularity of time*" [5].

Multiple models have been tested on fictive and real data. Yet, most of the research deals with rather small railway networks when considering the infrastructure at a microscopic level. Hence, there is a lack of knowledge on the performances of models on larger areas.

There is little research on the use of Constraint Programming (CP) to solve TTR problems, even though Bartak et al. [2] showed that CP is appropriate to model real-life scheduling applications. Rodriguez proposes a first model [20] for rerouting and reordering of trains at junctions. This model gives nice performances and has been improved lately by Cappart and Schaus [5, 6].

The traditional Constraint-Based Scheduling approaches commonly yield models composed of integer variables. Laborie et al. [14, 15] introduce a new type of variables, conditional time-intervals, which are purposely built for scheduling, with the intervals being optional by nature. The model by Cappart and Schaus [5, 6] makes use of this type of variables as well as their associated constraints. It also shows that these variables provide an elegant way to model this type of scheduling problems. Our work follows this trend as our model is directly based on their work.

Chapter 3

Constraint programming

This chapter introduces the concept of constraint programming. It presents the two components of the paradigm then demonstrates how it can be extended to optimization problems.

3.1 Principle

Constraint programming (CP) is a programming paradigm that is composed of two main components, the modeling and the search.

3.1.1 Modeling

Modeling aims at representing a mathematical problem, and is done using variables and constraints. Variables have an initial domain, and constraints state relations between these variables, or restrictions on values a variable can take. An example of variables is given in Equation 3.1, and a constraint over these variables is given in Equation 3.2.

$$dom(i) = \{0, 1, 2, 3\}, \quad dom(j) = \{2, 3\} \quad (3.1)$$

$$i \neq j \quad (3.2)$$

The goal of a **CP Solver** is to bind each variable to one value from their domain while respecting every constraint that is expressed over them.

3.1.2 Search

Search is carried in two steps: propagation and branching.

The propagation consists of the removal of inconsistent values from the domain of each variable. Consistency can be defined in multiple ways (*e.g.* bound consistency or domain consistency). The consistency level is a trade off between the propagation speed and the level of filtering that is made on the domains. For instance, bound consistency is reached faster than domain consistency but removes fewer values from the domains of the variables.

Branching consists of taking decisions on variables in order to explore the search space. When a decision is made, propagation takes place and reverberates that decision onto domains of the other variables. If this results in an inconsistency, *e.g.* a domain being emptied, the branching algorithm backtracks and takes another decision.

Branching can be done multiple ways, some of them represented in Figure 3.1. Decisions are generally guided by two heuristics.

1. Variable heuristic: which variable the search should select next ;
2. Value heuristic: which value of the variable the branching should act upon (bind the variable to it or split the domain based on it, for instance).

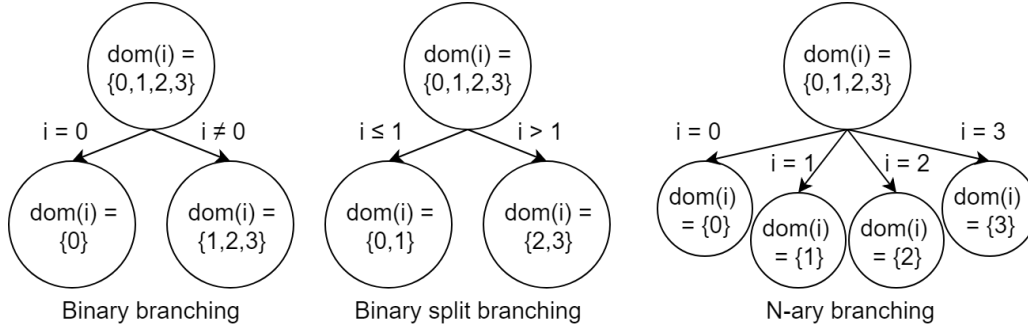


Figure 3.1: Examples of branching decisions

This branching can be automatic, with the practitioner only having to model its problem and the search being carried automatically, applying techniques that are selected using machine learning techniques, for instance. This is the case for CP Optimizer [13], where conditional time-interval variables, one of the main focus of this thesis, were first introduced by Laborie et al. [14, 15]. Obviously, CP Optimizer also lets the user use a customized search.

In some other solvers, branching has to be explicitly described by the user. This is the case for Oscar [18]. It is therefore possible to specialize the search for the problem at hand. The branching can be written from scratch, but some often-used techniques are directly available as well.

3.2 Constraint optimization problem

Constraint programming can also be used in optimization problems. A problem where the only goal is to find a feasible solution is called a constraint satisfaction problem (CSP), while a problem where the goal is to optimize some objective is called a constraint optimization problem (COP). For this, the algorithm is given some objective o . In the case of minimization, when the search finds a feasible solution where $o = o_1$, it is restarted with a new constraint $o < o_1$. Each time it finds a solution, it tightens the objective until the whole search space is explored or a stopping criterion is met.

Chapter 4

Conditional time-interval variables

This chapter presents a type of variables for constraint programming that were first introduced by Laborie and Rogerie [14], conditional time-interval variables. These variables facilitate the modeling of a range of scheduling problems. After an overview of the general principle, a selection of possible constraints is presented, followed by an introduction to the propagation on these constraints.

4.1 General principle

A conditional time-interval variable represents the time-interval of an activity whose execution status can take two values :

1. Executed: the activity represented by the variable is executed ;
2. Not executed: the activity is not executed and constraints do not consider the variable.

More formally, a time-interval variable v has a domain $\{[s, e) \mid s, e \in \mathbb{Z}, s \leq e\}$, where s is an integer that stands for the start time of the activity, and e is an integer that stands for its end time. This is in the case of a mandatory activity, *i.e.* an activity that has to be executed.

A *conditional* time-interval variable i has a domain $\{\perp\} \cup \{[s, e) \mid s, e \in \mathbb{Z}, s \leq e\}$ and takes the value

- $i = \perp$ if the activity is not executed ;
- $i = [s, e)$ if the activity is executed, starts at time s and ends at time e .

Hereafter, conditional time-interval variables will be referred to as CTI variables. The execution status of a CTI variable i is denoted $x(i)$ and can take value 0 (not executed) or 1 (executed). Without other information, $x(i)$ means that the variable is executed, and $\neg x(i)$ means that it is not. Similarly, the start and end of i are denoted $s(i)$ and $e(i)$ and can take any of the values from their initial domain.

4.2 Constraints

These variables are constituted of two dimensions: logical (execution status) and temporal (start and end times).

Constraints on CTI variables can therefore be classified according to these categories: constraints on the logical dimension, and constraints on the temporal one. Some hybrid constraints also restrict values from both dimensions. Most of these constraints and their semantic were introduced along with the CTI concept by Laborie and Rogerie [14].

4.2.1 Logical constraints

These constraints restrict the execution status of the variables that take part in it.

Execution

This constraint simply states that the activity corresponding to the interval has to be executed.

$$exec(i) \leftrightarrow x(i) = 1$$

Implication

This binary constraint between CTI variables i and j states that, if activity i is executed, activity j has to be executed.

$$implication(i, j) \leftrightarrow x(i) = 1 \implies x(j) = 1$$

Equivalence

This global constraint states that each CTI variable $\{i_1, \dots, i_n\}$ it constrains has to have the same execution status.

$$equivalence(i_1, \dots, i_n) \leftrightarrow x(i_1) \equiv \dots \equiv x(i_n)$$

4.2.2 Temporal constraints

These constraints restrict the time boundaries of the variables, and take only variables that are executed into account.

Start at start

This constraint between CTI variables i and j states that, if both activities are executed, i has to have the same start time as j .

$$startAtStart(i, j) \leftrightarrow x(i) \wedge x(j) \implies s(i) = s(j)$$

It is also possible to specify a gap value $z \in \mathbb{Z}$ such that

$$startAtStart(i, j, z) \leftrightarrow x(i) \wedge x(j) \implies s(i) + z = s(j)$$

The next few constraints follow the same pattern.

Start at end

$$\begin{aligned} startAtEnd(i, j) &\leftrightarrow x(i) \wedge x(j) \implies s(i) = e(j) \\ startAtEnd(i, j, z) &\leftrightarrow x(i) \wedge x(j) \implies s(i) + z = e(j) \end{aligned}$$

End at start

$$\begin{aligned} endAtStart(i, j) &\leftrightarrow x(i) \wedge x(j) \implies e(i) = s(j) \\ endAtStart(i, j, z) &\leftrightarrow x(i) \wedge x(j) \implies e(i) + z = s(j) \end{aligned}$$

End at end

$$\begin{aligned} endAtEnd(i, j) &\leftrightarrow x(i) \wedge x(j) \implies e(i) = e(j) \\ endAtEnd(i, j, z) &\leftrightarrow x(i) \wedge x(j) \implies e(i) + z = e(j) \end{aligned}$$

These constraints also exist in less restrictive forms, in which the strict equality between time boundaries is replaced by an ordered inequality relation.

Start before start

$$\begin{aligned} startBeforeStart(i, j) &\leftrightarrow x(i) \wedge x(j) \implies s(i) \leq s(j) \\ startBeforeStart(i, j, z) &\leftrightarrow x(i) \wedge x(j) \implies s(i) + z \leq s(j) \end{aligned}$$

Start before end

$$\begin{aligned} startBeforeEnd(i, j) &\leftrightarrow x(i) \wedge x(j) \implies s(i) \leq e(j) \\ startAtEnd(i, j, z) &\leftrightarrow x(i) \wedge x(j) \implies s(i) + z \leq e(j) \end{aligned}$$

End before start

$$\begin{aligned} endBeforeStart(i, j) &\leftrightarrow x(i) \wedge x(j) \implies e(i) \leq s(j) \\ endBeforeStart(i, j, z) &\leftrightarrow x(i) \wedge x(j) \implies e(i) + z \leq s(j) \end{aligned}$$

End before end

$$\begin{aligned} endBeforeEnd(i, j) &\leftrightarrow x(i) \wedge x(j) \implies e(i) \leq e(j) \\ endBeforeEnd(i, j, z) &\leftrightarrow x(i) \wedge x(j) \implies e(i) + z \leq e(j) \end{aligned}$$

No overlap

This global constraint on $\{i_1, \dots, i_n\}$ is useful for unary resources that can handle at most one activity at a given time and is introduced by Laborie et al. [15]. It enforces the fact that there cannot be any overlap between executed intervals in $\{i_1, \dots, i_n\}$.

$$noOverlap(i_1, \dots, i_n) \leftrightarrow \forall (j, k)_{j, k \in [1, n] \wedge j \neq k} \wedge x(j) \wedge x(k) \quad e(i_j) \leq s(i_k) \vee e(i_k) \leq s(i_j)$$

Specifying a transition time between activities is also possible via a transition time matrix M . The constraint is then

$$\text{noOverlap}(i_1, \dots, i_n, M) \leftrightarrow \forall (j, k)_{j, k \in [1, n] \wedge j \neq k \wedge x(j) \wedge x(k)} \quad e(i_j) + M(j, k) \leq s(i_k) \vee e(i_k) + M(k, j) \leq s(i_j)$$

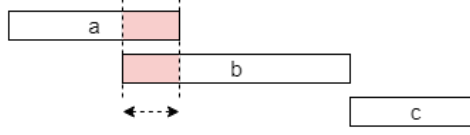


Figure 4.1: Overlap between activities a and b

4.2.3 Hybrid constraints

Span

This global constraint on i and $\{j_1, \dots, j_n\}$ enforces restrictions on logical and temporal values of the variables that are involved in it. It states that, if i is executed, then i starts with the earliest starting executed variable in $\{j_1, \dots, j_n\}$ and ends with the latest ending executed variable in $\{j_1, \dots, j_n\}$. i is not executed if and only if no variable in $\{j_1, \dots, j_n\}$ is executed.

$$\text{span}(i, \{j_1, \dots, j_n\}) \leftrightarrow \begin{cases} \neg x(i) \Leftrightarrow \forall k \in [1, n] \neg x(j_k) \\ x(i) \Leftrightarrow \begin{cases} \exists k \in [1, n] \quad x(j_k) \\ s(i) = \min_{k \in [1, n]} s(j_k) \\ e(i) = \max_{k \in [1, n]} e(j_k) \end{cases} \end{cases}$$

Alternative

This constraint on i and $\{j_1, \dots, j_n\}$ is similar to the span constraint apart from the fact that i is executed if and only if exactly one variable in $\{j_1, \dots, j_n\}$ is executed. The time boundaries of i and that executed variable are then equal.

$$\text{alternative}(i, \{j_1, \dots, j_n\}) \leftrightarrow \begin{cases} \neg x(i) \Leftrightarrow \forall k \in [1, n] \neg x(j_k) \\ x(i) \Leftrightarrow \exists k \in [1, n] \begin{cases} x(j_k) \\ s(i) = s(j_k) \\ e(i) = e(j_k) \\ \forall l \neq k \neg x(j_l) \end{cases} \end{cases}$$

4.3 Propagation

The optional nature of these activities implies that the propagation procedures of each of the above constraints have to be adapted. The naive solution is to only propagate domain reductions or value binding of temporal boundaries of a variable if it is assured to be executed ($x(i) = 1$). However, this means that the execution status of a variable has to be bound before any domain reduction on its temporal dimension can happen.

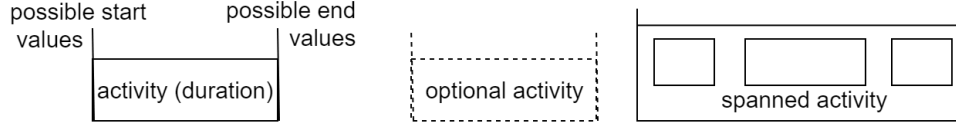


Figure 4.2: Representation of type of activities

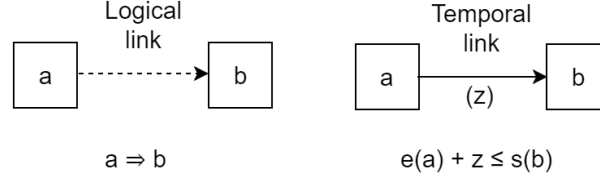


Figure 4.3: Representation of links between activities

An example of a case is represented in Figure 4.4. The activities and their links are graphically represented according to the convention depicted in Figures 4.2 and 4.3.

It represents a case where the main activity is to wash clothes. This activity is mandatory and is therefore pictured as a box with a solid border. Possible values for start and end boundaries are written at each extremity of the box.

There are two possibilities for this activity: hand wash the clothes or put them in the washing machine. This is represented by the XOR link between them and the main activity. As these alternatives are not mandatory, they are depicted as boxes with dotted borders. Furthermore, the hand wash option is composed of several sub-activities (span constraint), where every sub-activity has to be executed if the spanned activity is executed (represented by the logical links between the spanned activity and its components). These sub-activities have a fixed duration and are linked with temporal restrictions (end before start constraints), as they have to be executed in that particular order.

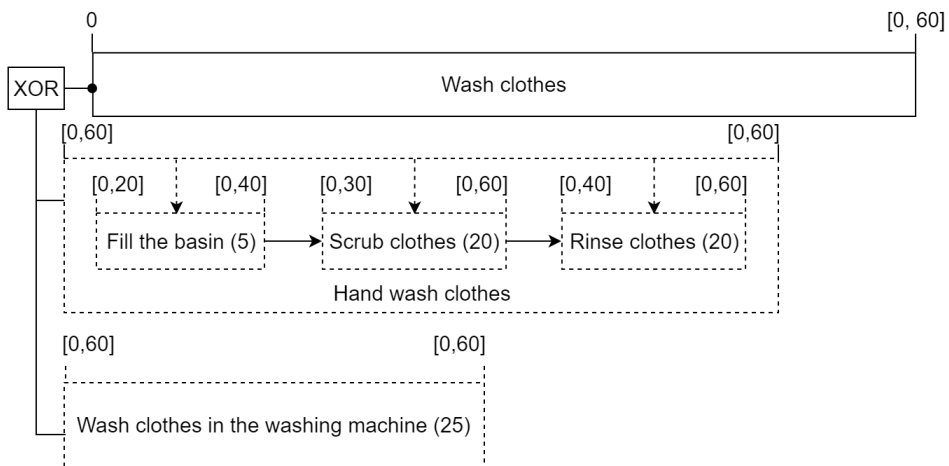


Figure 4.4: Initial state of the activities

The naive propagator that only takes executed activities into account cannot reduce a single domain in this situation, as only one activity is assured to be executed. However, a lot of information can be exploited in order to propagate and get the result shown in Figure 4.5.

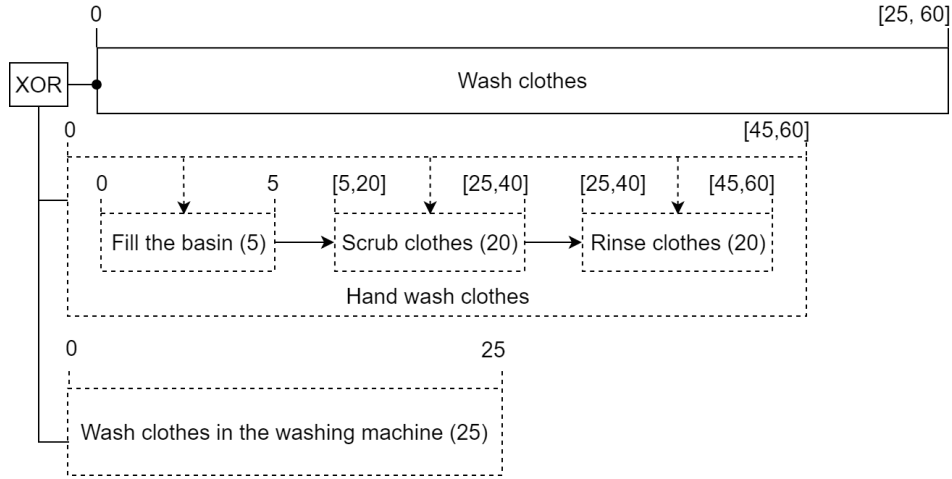


Figure 4.5: State of the activities after propagation

This result can be achieved by the addition of two network-like structures that help the propagation, a logical and a temporal network, as described by Laborie and Rogerie [14].

Logical network

This structure is a graph that contains information on links between execution status of CTI variables. For variable i , two nodes are possible: $x(i)$ and $\neg x(i)$. An edge between nodes n and m states that there exists an implication $n \implies m$. For the propagation to be possible, the network needs to have these properties:

- Detection of inconsistencies in the network (when the network infers $n \implies \neg n$ and $\neg n \implies n$) ;
- Access to the logical relation between any pair of activities ;
- Access to the activities implied by an activity, and to the activities that imply an activity ;
- Detection of new logical relations and the activation of events accordingly.

Temporal network

As its logical counterpart, this is also a graph, which contains information on temporal links between activities. The nodes of this graph are temporal boundaries of activities, *i.e.* for an activity i , two nodes will be present in the graph: $s(i)$ and $e(i)$. This graph is weighted, and an edge of weight w between two nodes n and m corresponds to the relation $n + w \leq m$.

To propagate efficiently, the temporal network takes advantage of the logical relations inferred by the logical network.

Chapter 5

CTI variables for OsaR

This chapter proposes an implementation of CTI variables for OsaR [18], along with the components they rely on. OsaR is a toolkit that aims at solving operational research problems. It is a Scala library, both written and usable with the Scala language. OsaR has multiple components, one of which is a constraint programming solver. This is the part that this chapter will focus on extending [19]. Firstly, the implementation of CTI variables is given. Secondly, both logical and temporal networks implementations are explained, and then the constraints that take advantage of them. As search is an important part of CP, it will also be explored in this chapter, followed by some performance metrics, in particular compared to models using reified variables.

5.1 Anatomy of a conditional time-interval

As a reminder, a CTI is composed of the following elements:

- an execution status (0 for not executed, 1 for executed) ;
- a start domain, that contains every possible start time value ;
- an end domain, that contains every possible end time value ;
- a duration domain, that contains every possible duration value.

The variable is bound if either

- the execution status is bound to 0 or
- the execution status is bound to 1, the start is bound to a value, the end is bound to a value such that $start \leq end$, and the duration is bound to a value such that $duration = end - start$. Each of these values should of course be part of the original domain of the variable.

OsaR, similarly to other CP solvers, is primarily centered around integer variables. The class `CPIntVar`¹ proposes an implementation of these integer variables that is adaptive to the type of domain the variable is defined on (*e.g.* singleton, continuous domain, sparse domain). The implementation of each method is then optimized depending on the domain type. `CPIntVar` is also extended by `CPBoolVar`, which corresponds to boolean variables that can only take values

¹The class `CPIntVar` is actually abstract ; for simplicity, it is considered here as a complete implementation.

0 or 1. These two types of variables correspond therefore very well to the components of a CTI variable.

However, a normal integer variable cannot have its domain emptied, be it by the propagation or by the tree search of the solver. Indeed, this throws an `Inconsistency` exception which either triggers a `NoSolutionException` if it happens during the first propagation steps, before the tree search, or tells the tree search to backtrack and make other choices. This cannot happen in this case, because an emptied domain for the start, end or duration sub-variables of the CTI does not mean the search has to be stopped ; it only means the execution status of the CTI variable has to be assigned to *false*, or 0.

For this to be possible, a new implementation of the `CPIntVar` class has to be created. It is called a `CPTimePointVar` and represents the temporal kind of sub-variables of the CTI (start, end, duration). It consists of a wrapper for a `CPIntVar` and takes advantage of the automatic implementation selection depending on the type of domain. The only difference is that it catches unwanted `Inconsistency` exceptions and assigns the execution status of the associated CTI to 0 in place. This structure is depicted in a simplified class diagram in Figure 5.1.

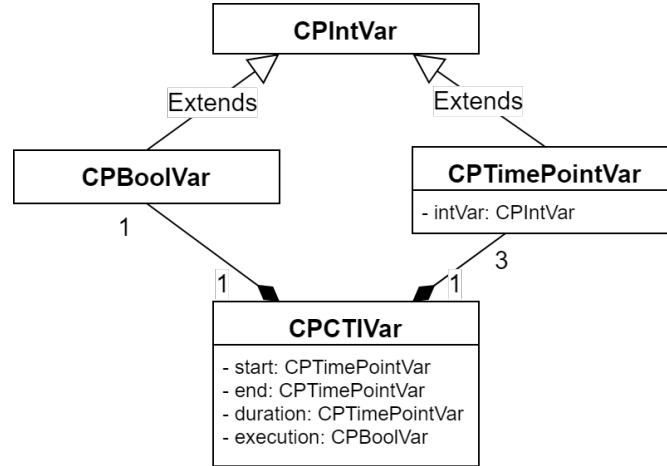


Figure 5.1: Composition of a CTI variable

The setup of a CTI variable is composed of multiple steps:

1. Create the appropriate `CPTimePointVar` variables for start, end and duration ;
2. Add a constraint $start \leq end$;
3. Add a constraint on the duration for it to be equal to $end - start$;
4. Add two edges in the temporal network that link the three temporal sub-variables in this way:

$$start + \min(duration) \leq end$$

$$end - \max(duration) \leq start$$

That corresponds, for a CTI i , to links $s(i) \xrightarrow{\min d(i)} e(i)$ and $e(i) \xrightarrow{-\max d(i)} s(i)$. The weight of the edges are of course updated when the duration domain changes.

Documentation on `CPCTIVar` variables in *Oscar* is available in Appendix A.

5.2 Logical network

As explained above, the logical network is a graph where the nodes are executions of CTI or their negation. A link for CTI variables i and j in the graph $x(i) \rightarrow \neg x(j)$ states that there is a logical implication from $x(i)$ to $\neg x(j)$. This logical network is constructed incrementally, adding nodes and vertices when new constraints are added.

Because the network evolves depending on choices that are made during the search phase, the structure has to be reversible. This means that it has to be able to be reversed to an earlier state, to make backtracking feasible. Most of the structures in Oskar are actually reversible. This concerns variables domains, but also sets, integers or boolean variables just to name a few. Two options are possible for the implementation of this graph. From now on, $|V|$ and $|E|$ respectively refer to the number of vertices and edges in a graph.

Firstly, it is possible to use an adjacency matrix M , in which each component is a reversible boolean, where $M(x, y) = 0$ means there is no implication between x and y , and $M(x, y) = 1$ means there is an implication $x \implies y$. This reuses existing structures and the time complexity of the insertion of a new vertex can be reduced from $O(|V|^2)$ (property of adjacency matrices) to an amortized linear time by the use of Scala `ArrayBuffer`² structures for the matrix. However, while the constant access time to a relationship between two CTI variables is what is needed, the memory usage ($O(|V|^2)$) and queries for successors or predecessors ($O(|V|)$) are too resource-intensive, as the graph is mostly sparse in the majority of the cases.

The second option is therefore to use another type of data structure, adjacency lists, or rather sets. The network is composed of an `ArrayBuffer` L containing reversible sets, to ensure possible backtracking. $L(i)$ containing j means that there is a link $i \implies j$. The `ArrayBuffer` ensures an amortized constant time $O(1)$ for vertex addition, while the set ensures an $O(1)$ time complexity for edge addition. A comparison of the important metrics is available in Table 5.1.

Table 5.1: Comparison between possible structures for the temporal network

	Adjacency set	Adjacency matrix
Memory	$O(V + E)$	$O(V ^2)$
Add vertex	$O(1)$	$O(V)$ (amort.)
Add edge	$O(1)$	$O(1)$
Remove vertex	$O(E)$	$O(V ^2)$
Adjacency query	$O(1)$	$O(1)$

However, because the existing reversible set in Oskar does not have a *contains* function, an extension that implements it is mandatory to reduce the adjacency query time from $O(|V|)$ down to $O(1)$. The network therefore consists of an `ArrayBuffer` containing extensions of reversible sets that implement the *contains* method. Sets containing predecessors for each node are also used for faster access of predecessors and faster removal of vertices.

Each node is given an index at its creation and the correspondences $node \rightarrow index$ and $index \rightarrow node$ can be established thanks to auxiliary `HashMap` structures. The main purpose of this is to index the `ArrayBuffer`, and to take advantage of Oskar's propagation procedures that

²`ArrayBuffer` structures are used because the networks are built iteratively ; the number of nodes in each network is not known in advance.

can take an integer as parameter.

An example of how a constraint $x(i) \Rightarrow \neg x(j)$ is stored in the network is shown in Figure 5.2.

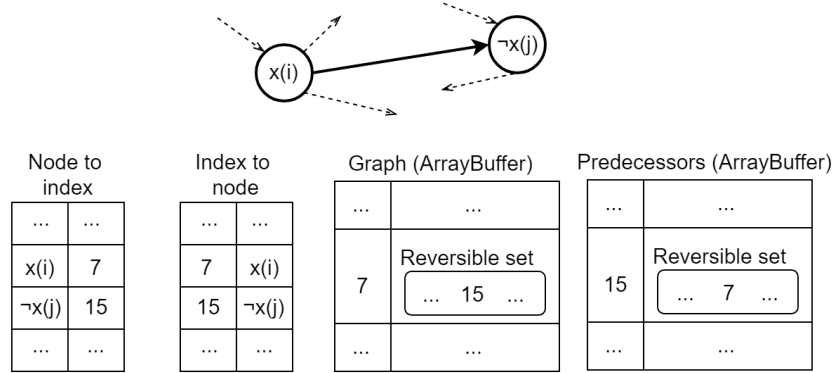


Figure 5.2: Representation of the link $x(i) \rightarrow \neg x(j)$ in the logical network

The network operations are explained below.

Adding a link

Adding a link $i \Rightarrow j$ to the logical network is done in two steps. To avoid unnecessary operations, the procedure is not carried if the link already exists.

1. Create the nodes i and j in the graph if they are not present yet, and add an edge from i to j ;
2. Add i to the set of predecessors of j .

The ArrayBuffer containing each set is accessed via an index that is remembered by the auxiliary HashMap mentioned above. Sets are used because the order does not matter and duplicate edges are not possible. This means that:

- access to the activities implied by an activity is done in $O(|E|)$ (just iterate the corresponding set in the graph) ;
- access to the activities another activity implies is done in $O(|E|)$ (iterate the *predecessors* set) ;
- adding an edge is done in amortized constant time, because it consists of creating the nodes in the graph (amortized constant time due to the ArrayBuffer), adding an element to the appropriate set in the graph ($O(1)$), and adding an element to a *predecessors* set ($O(1)$).

However, for the network to respect the properties cited in Section 4.3, detection of inconsistencies and access to the logical relation between any pair of activities are still needed. As the implication relation is transitive, the latter can be achieved via a multitude of graph algorithms. It is however not efficient enough as relations between variables are accessed a large number of times in this case. The ideal solution is to have an access that can be done in $O(1)$.

Algorithm 1 The collapse procedure

```

function COLLAPSE( $s$ : a sequence of nodes to collapse)
   $connectedComponents \leftarrow$  global connected components map
   $collapsedNodes \leftarrow$  global collapsed nodes set
   $head \leftarrow$  first node of  $s$ 
  if  $head \notin connectedComponents$  then
     $set \leftarrow$  new ReversibleSet
  else
     $set \leftarrow connectedComponents(head)$ 
  end if
  for all  $i \in s, i \neq head$  do
    add  $i$  to  $set$ 
    remove  $i$  from  $connectedComponents$  ▷ Remove references to internal SCC's
    add  $i$  to  $collapsedNodes$ 
  end for
  add  $(head, set)$  to  $connectedComponents$ 
end function

```

Processing new links

Firstly, every new link is checked for inconsistency: if the link is $x(i) \rightarrow \neg x(i)$, we check that $\neg x(i) \rightarrow x(i)$ is not already present in the network. Otherwise, an **Inconsistency** exception is thrown.

For the constant time access to any relation in the graph, an operation that is based on Brafman [3] is added. The transitive closure of a graph G is a graph G' where the edge (v, u) exists if there is a path from v to u in G . The idea is to compute the transitive closure of the network, *i.e.* to take implicit transitive relations and make them explicit. Access to a relation between two variables is therefore done in $O(1)$ (look up in a set).

It is easily computed in a directed acyclic graph, which is not the case here as for instance some relations can be mutual. There are therefore four steps to this operation: compute the strongly connected components of the graph, collapse them into a single node, compute the reverse topological order of the graph and then compute its transitive closure.

The **strongly connected components** are computed with the Tarjan [21] algorithm, which is linear and runs in $O(|V| + |E|)$.

The **collapse** method is then called on each of these strongly connected components and is described in Algorithm 1. The collapsing of the nodes is simulated by selecting a node to represent every other one in its connected component. They are linked with a HashMap, while the collapsed nodes are added to a set which is used to filter them out by the topological ordering DFS. The collapse method has a time complexity of $O(|E|)$.

Once the strongly connected components are collapsed, the graph is virtually acyclic while keeping information on every relation between any two variables. It is therefore possible to compute **the reverse topological order** of the graph. The topological order of a graph G is a sequence containing all its nodes (here only the ones that were not collapsed into others) where

the order $<_t$ is respected. $<_t$ is defined as

$$\forall (i \rightarrow j) \in G \quad i <_t j$$

i.e. i comes before j in the sequence if there exists a link $i \rightarrow j$ in the graph.

The reverse topological ordering is done by a method based on DFS and has a time complexity of $O(|V| + |E|)$.

The last step is to compute the **transitive closure**. This is done following the reverse topological order by adding, for each node, the nodes accessible from any of its children to its own children.

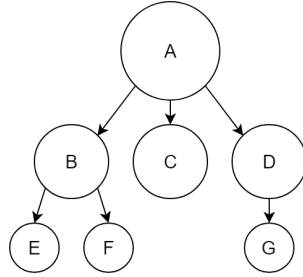


Figure 5.3: The initial direct successors of A are B, C and D

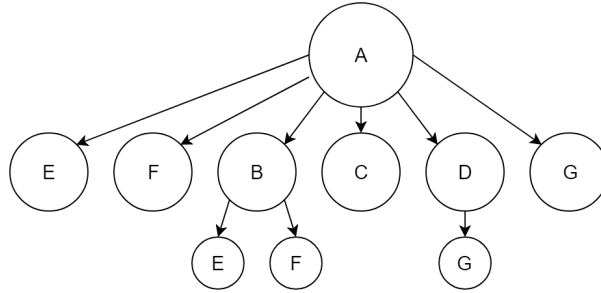


Figure 5.4: After the transitive closure, the direct successors of A are B, C, D, E, F and G

Figures 5.3 and 5.4 show the update of the transitive closure for one node A . As the procedure has to be done on every node, we know the time complexity to be $O(|V| \times |E|^2)$. The transitive closure is therefore a heavy procedure even if, according to Laborie and Rogerie [14], the size of the implication graph tends to be small compared to the number of variables in most scheduling problems. The transitive closure is consequently computed only after having added the largest possible number of nodes to the graph, as opposed to after each *add* operation, in order to keep the consequences of its complexity as low as possible.

Figure 5.5 depicts the two procedures schematically.

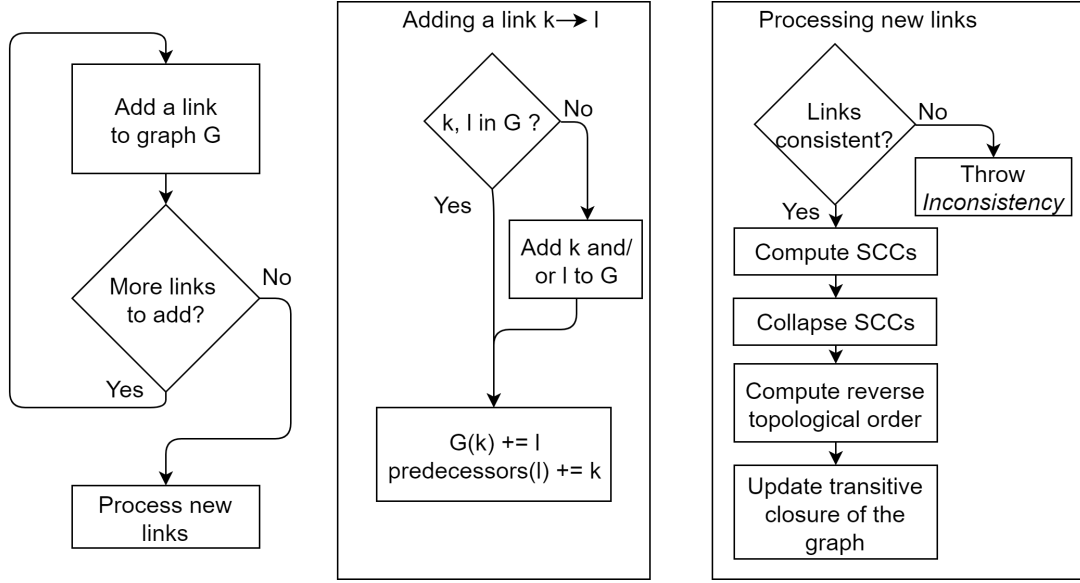


Figure 5.5: Summary of the two procedures in the logical network

Propagating execution status

When the execution status of a CTI variable i is bound, it is propagated onto the variables it is linked with on the logical network. For this, we check nodes in the network that correspond to the CTI variable: $x(i)$ and $\neg x(i)$. For both of these, we can infer some information about other nodes' propagation status.

The `match` procedure described in Algorithm 2 is key to this propagation. The propagation procedure is shown in Algorithm 3.

If the status of a variable i matches the node n , we can propagate on every logical link $n \rightarrow m$ as the link is semantically the same as $true \rightarrow m$. m can therefore be updated so that it is true.

If the status of a variable i does not match the node n , we can propagate on every link $l \rightarrow n$ as the link is semantically the same as $l \rightarrow false$. l can therefore not be true.

For instance, if n represents $x(i)$ and i is not being executed, and l represents $\neg x(j)$, we have a link that states $\neg x(j) \Rightarrow false$ and we therefore have to assign the execution status of j to 1 in order to have $\neg x(j) = false$.

Algorithm 2 The match procedure

```

function MATCH( $i$ : CTI variable,  $n$ : node in the logical network)
  if execution status of  $i$  is true &  $n = x(i)$  then
    return true
  else if execution status of  $i$  is false &  $n = \neg x(i)$  then
    return true
  else
    return false
  end if
end function

```

The execution status propagation is done in $O(|E|)$. Once the execution status is propagated, the variable is virtually removed from the network, from its predecessors' *successors* set (the

Algorithm 3 The execution status propagation

```

function PROPAGATESTATUS(n: node in the logical network)
  ▷ As a reminder, a node in the logical network is a tuple (CTI variable, execution boolean)
  i ← CTI variable corresponding to node n
  if match(i, n) then
    for all successor m of n in the network do
      j ← CTI variable corresponding to node m
      jExecution ← execution corresponding to node m
      assign execution status of j to jExecution
    end for
  else
    for all predecessor m of n in the network do
      j ← CTI variable corresponding to node m
      jExecution ← execution corresponding to node m
      assign execution status of j to  $\neg jExecution$ 
    end for
  end if
end function

```

graph set) and from its successors' *predecessors* set, as nothing can be inferred from it anymore.

Triggers of events in the network

When it is added to the network, a node creates a trigger that states that when the execution status of the corresponding variable is bound, either by search or by propagation, it has to propagate this status to its successors and predecessors and then remove the node from the graph. Additionally, the temporal network sometimes detects temporal inconsistencies between variables and infers that they cannot be executed together. This constraint is then added to the logical network.

5.3 Temporal network

The temporal network is represented by a weighted directed graph. As a reminder, a node in the network corresponds to the temporal boundary of a CTI variable, either its start or its end. A link between two nodes *m* and *n* of weight *w* means that $m + w \leq n$.

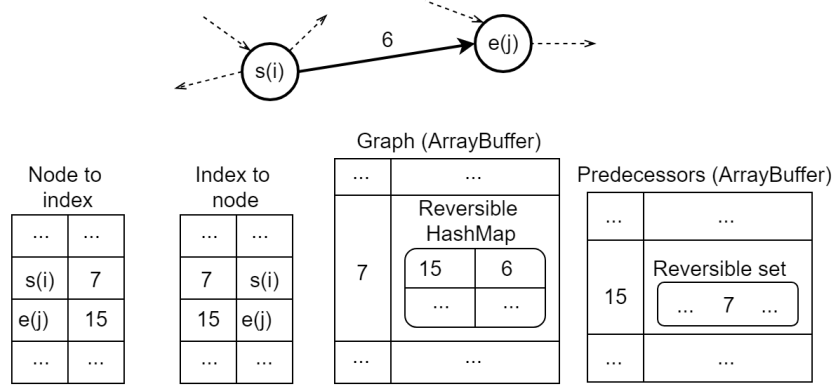
Structure

As before, the structure for the graph has to be reversible and adapted to sparse graphs. However, unlike the logical network, the temporal network has to be able to contain weights on its edges.

The network possesses therefore two main components:

- An ArrayBuffer containing a reversible HashMap ;
- A *direct predecessors* set.

As for the logical network, each node is given an index at its creation. Inside the ArrayBuffer, multiple HashMap structures maintain the links and the weights associated to it. An example is given in Figure 5.6, where the constraint $s(i) + 6 \leq e(j)$ is stored.

Figure 5.6: Representation of the link $s(i) \xrightarrow{6} e(j)$ in the temporal network

The operations on the network are explained below.

Adding a link

Firstly, the indexes are created for the nodes if they do not exist yet, then the weight of the edge is stored, and each node is added in the appropriate adjacency set of the other node. After this, the propagation starts with the two nodes as the initial points of propagation.

Propagation

The propagation is based on an algorithm described in Cesta and Oddi [7]. A simplified version of it, for clarity purposes, is presented in Algorithm 4. The main principle of it is that propagation is done on nodes that share a logical relation in the logical network and can be done in two ways, described in Laborie and Rogerie [14].

- For a link $i \xrightarrow{w} j$ where i logically implies j , it is sure that if i is executed, j will be executed. It is therefore possible to update the domain of i to be consistent with the domain of j . As $i \xrightarrow{w} j$ means $i + w \leq j$, the propagation updates the domain of i as such:

$$\max(i) = \min(\max(i), \max(j) - w)$$

- For a link $i \xrightarrow{w} j$ where j logically implies i , the logic is similar except that it is reversed. As $i \xrightarrow{w} j$ means $i + w \leq j$, the propagation updates the domain of j as such:

$$\min(j) = \max(\min(j), \min(i) + w)$$

- If the logical network contains both relations ($i \implies j$ and $j \implies i$), propagation is done both ways.

Algorithm 4 The temporal propagation procedure (simplified)

```

function PROPAGATE( $s$ : a sequence of nodes to be propagated)
   $Q \leftarrow$  queue from  $s$ 
  while  $Q$  is not empty do
     $u \leftarrow$  pop from  $Q$ 
    if  $u$  status execution is not false then
      for all successor  $v$  of  $u$  in the temporal graph do
        if  $v$  logically implies  $u$  then
           $w \leftarrow \text{graph}(u, v)$ 
          if  $\min(v) < \min(u) + w$  then
            update  $\min(v)$  to  $\min(u) + w$ 
            add  $v$  to  $Q$ 
          end if
        end if
      end for
      for all predecessor  $t$  of  $u$  in the temporal graph do
        if  $t$  logically implies  $u$  then
           $w \leftarrow \text{graph}(t, u)$ 
          if  $\max(t) > \max(u) - w$  then
            update  $\max(t)$  to  $\max(u) - w$ 
            add  $t$  to  $Q$ 
          end if
        end if
      end for
    end if
  end while
end function

```

As stated, the presented algorithm is a simplified version. Some elements are added to it in order for it to be complete and more efficient:

- Weights on links between the same CTI variable are updated during the propagation, as the weight represents its duration, whose domain changes during the execution.
- Inconsistent relations are added to the logical network. A relation between a and b where $\min(a) + w_{(a,b)} > \max(b)$ is inconsistent and links are added to the logical network to state that executions of a and b are incompatible.
- A cache is used to memoize the last value that was propagated for each node. It is not propagated again until that value has changed, unless some logical relations have changed that involve the same variable.
- The queue that guides the propagation is accompanied by a set to make sure every element is in the queue only once.

The propagation has a time complexity of $O(|V|^2)$.

Triggers of events in the network

When it is added to the network, each CTI variable adds a trigger to the store that states that when its domain is updated, propagation should take place starting from the corresponding node.

In addition, when the logical network infers some new relations, it tells the temporal network to propagate the nodes corresponding to the variables involved in it as well. The propagation triggers for both networks are depicted in Figure 5.7. The triggers on each network only actually start propagation if their value cache is not up to date.

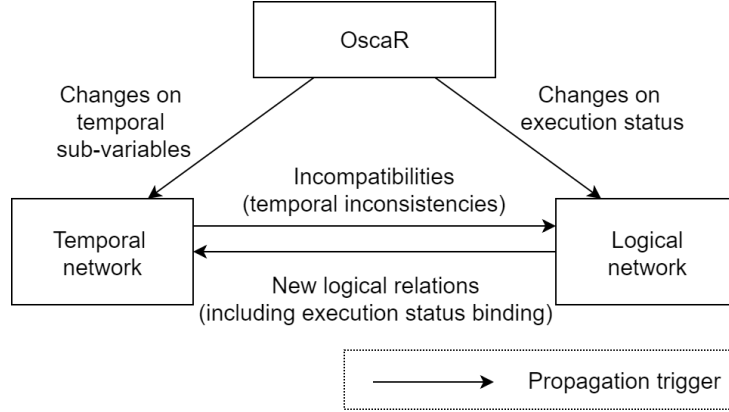


Figure 5.7: Propagation triggers on the networks

5.4 Constraints

This section explains how the constraints are implemented in OscanR and how they take advantage of both networks. The networks are created along with the first `CPCTIVar`, and links are added during the instantiation of variables or the addition of constraints. The networks are therefore invisible to the user, which eases the modeling.

CPCTIVar

CTI variables restrict, by nature, the values each of their component can take. It therefore constrains these components for them to be consistent. As stated in Section 5.1, four constraints are added:

- $start \leq end$
- $duration = end - start$
- $start + min(duration) \leq end$ in the temporal network
- $end - max(duration) \leq start$ in the temporal network

Execution

This constraint is the most basic one and is directly implemented in the CTI variable, which contains both `assignTrue()` and `assignFalse()` methods, which respectively assign the execution status of the variable to true or false.

Implication

The implication constraint is easily represented on the logical network and therefore consists of the addition of the link $x(i) \implies x(j)$ on the network. For `CPCTIVar` `a` and `CPCTIVar` `b`, there are two ways of adding it to the store:

$$\text{add}(\text{Implication}(a, b))$$

$$\text{add}(a \implies b)$$

Equivalence

For this global constraint, involved variables are treated two by two. For each pair of variables (i, j) , two links are added to the logical network :

$$x(i) \implies x(j)$$

$$x(j) \implies x(i)$$

Here also, two ways of adding it to the store are available. For an equivalence between n variables:

$$\text{add}(\text{Equivalence}(\text{Array}(i_1, \dots, i_n)))$$

For an equivalence between two variables:

$$\text{add}(i \iff j)$$

Binary temporal constraints

Table 5.2: Binary temporal constraints

Constraint	Links in the temporal network	Addition to the store
Start at start	$s(a) \xrightarrow{z} s(b)$ $s(b) \xrightarrow{-z} s(a)$	$\text{add}(\text{StartAtStart}(a, b, z))$ $\text{add}(\text{StartAtStart}(a, b))$
Start at end	$s(a) \xrightarrow{z} e(b)$ $e(b) \xrightarrow{-z} s(a)$	$\text{add}(\text{StartAtEnd}(a, b, z))$ $\text{add}(\text{StartAtEnd}(a, b))$
End at start	$e(a) \xrightarrow{z} s(b)$ $s(b) \xrightarrow{-z} e(a)$	$\text{add}(\text{EndAtStart}(a, b, z))$ $\text{add}(\text{EndAtStart}(a, b))$
End at end	$e(a) \xrightarrow{z} e(b)$ $e(b) \xrightarrow{-z} e(a)$	$\text{add}(\text{EndAtEnd}(a, b, z))$ $\text{add}(\text{EndAtEnd}(a, b))$
Start before start	$s(a) \xrightarrow{z} s(b)$	$\text{add}(\text{StartBeforeStart}(a, b, z))$ $\text{add}(\text{StartBeforeStart}(a, b))$
Start before end	$s(a) \xrightarrow{z} e(b)$	$\text{add}(\text{StartBeforeEnd}(a, b, z))$ $\text{add}(\text{StartBeforeEnd}(a, b))$
End before start	$e(a) \xrightarrow{z} s(b)$	$\text{add}(\text{EndBeforeStart}(a, b, z))$ $\text{add}(\text{EndBeforeStart}(a, b))$
End before end	$e(a) \xrightarrow{z} e(b)$	$\text{add}(\text{EndBeforeEnd}(a, b, z))$ $\text{add}(\text{EndBeforeEnd}(a, b))$

To constrain a CTI variable a to start at the same time as another variable b , with an optional delay z , two links are added to the temporal network:

$$s(a) \xrightarrow{z} s(b)$$

$$s(b) \xrightarrow{-z} s(a)$$

It is added to the store with

$$\text{add}(\text{StartAtStart}(a, b, z)) \text{ or}$$

$$\text{add}(\text{StartAtStart}(a, b)) \text{ if there is no delay}$$

Obviously, every constraint of the form [Start|End] [Before|At] [Start|End] follows the same pattern, which is detailed in Table 5.2.

The following three constraints have propagation procedures on their own, sometimes in addition to propagation in the temporal and logical networks.

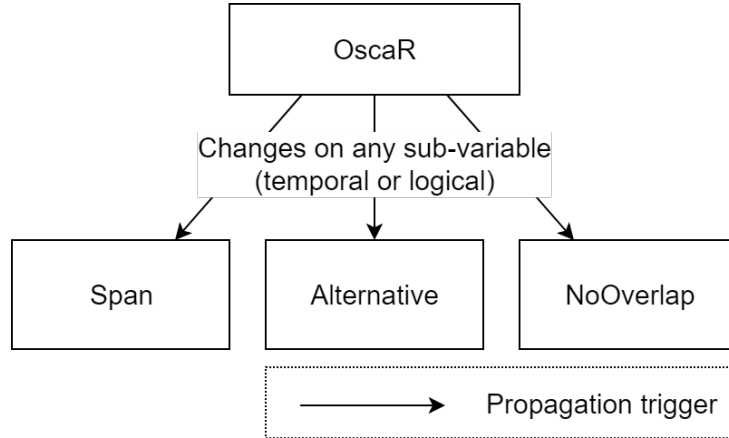


Figure 5.8: Propagation triggers on the constraints

No overlap

The no overlap constraint, as opposed to the above constraints, has a propagate method that is proper to it. It is based on three principles:

- For CTI a and CTI b , if $\min e(b) > \max s(a)$, a will always precede b ;
- For CTI a and CTI b , they always overlap if neither these conditions is respected:

$$\max(\min s(a), \max s(b)) \geq \min(\min e(a), \max e(b))$$

$$\max(\min s(b), \max s(a)) \geq \min(\min e(b), \max e(a))$$

- If overlap is detected between two activities, and the logical relation between them means that they will both be executed, an **Inconsistency** exception is thrown.

A simplification of the propagation procedure is presented in Algorithm 5 and is called for every variable during the constraint setup. During the execution, it is called when temporal boundaries of involved variables change or when their execution status is bound. The constraint holds in memory the remaining active variables in a set (*i.e.* the ones that still have a chance of being executed) and the preceding constraints it has already added to the store in order to reduce the number of operations at each run of the propagation procedure.

Algorithm 5 Simplified NoOverlap propagation procedure

```

function PROPAGATE(activityToPropagate: CPCTIVar)
  if activityToPropagate execution status is false then
    remove activityToPropagate from the set of variables in the constraint
    if size of the set of variables in the constraint  $\leq 1$  then
      deactivate the constraint
    end if
  else
    for all activity in the constraint, activity  $\neq$  activityToPropagate do
      if activityToPropagate logically implies activity
        & activityToPropagate execution status is true
        & activityToPropagate has to overlap with activity given their domains then
          throw Inconsistency exception
        end if
      if activityToPropagate has to precede activity given their domains then
        add EndBeforeStart(activityToPropagate, activity) to the store
      end if
      if activity precedes automatically activityToPropagate then
        add EndBeforeStart(activity, activityToPropagate) to the store
      end if
    end for
  end if
end function

```

Span

The span constraint between i and $\{j_1, \dots, j_n\}$ is handled by both logical and temporal networks. For each pair (i, j_k) , a link $x(j_k) \rightarrow x(i)$ is added to the logical network, and two temporal constraints are added:

$$StartBeforeStart(i, j_k)$$

$$EndBeforeEnd(j_k, i)$$

However, an additional specialized propagation procedure is needed. The procedure makes sure that if i is executed, there exists at least one activity it spans onto that is executed and starts at the same time as i , and at least one activity it spans onto that is executed and ends at the same time as i . It also throws an **Inconsistency** exception when i is executed but every other variable involved in the constraint is not executed.

Alternative

Similarly to the span constraint, the alternative constraint between i and $\{j_1, \dots, j_n\}$ is handled by both networks. For each pair (i, j_k) , a link $x(j_k) \rightarrow x(i)$ is added to the logical network, and two temporal constraints are added:

$$StartAtStart(i, j_k)$$

$$EndAtEnd(j_k, i)$$

On top of this, for each distinctive pair (j_k, j_l) , two links are added to the logical network to state that their executions are incompatible:

$$x(j_k) \rightarrow \neg x(j_l)$$

$$x(j_l) \rightarrow \neg x(j_k)$$

The propagation procedure makes sure that if i is executed, then one of the alternative j_k is also executed. It also updates the bounds of i . The minimum start of i becomes the minimum of each possible alternative start, and the maximum end of i becomes the maximum end of each possible alternative end.

5.5 Search

Search is one of the main parts of constraint programming. As explained in Section 3.1.2, in OsaR, the user has to specify the search and specialize it to the problem. However, some built-in branching algorithms for `CPIntVar` variables are available and can be used for `CPCTIVar` variables as well, as they are composed of `CPIntVar` sub-variables. An implicit conversion is implemented to make the available search methods compatible with `CPCTIVar` variables directly. In this section, three branching schemes are explored: first fail search, last conflict (LC) search and conflict ordering (CO) search. Then, some variable and value ordering heuristics are given.

5.5.1 Branching schemes

First fail search

This search implements the *fail-first* principle described by Haralick and Elliott [12] which states "*To succeed, try first where you are most likely to fail.*". This principle is widely used for variable ordering heuristics. Therefore, this search orders the variables by the size of their domain, and starts with the variables having the smallest domain. The variable ordering evolves along the sizes of the domains during the search. In most cases, for `CPCTIVar` variables, the execution status of the variables will be selected first, as their domain contains at most two elements. The value selection heuristic is either chosen by the user or defaults to the minimum of the domain. This search exists in both binary (two branches: $i = i_1, i \neq i_1$) and in split (two branches: $i \leq i_1, i > i_1$) forms. In Section 5.6, only the binary form of the search is tested.

Last conflict search

Introduced by Lecoutre et al. [16], this search aims at avoiding to explore similar subtrees that yield no solutions. It detects decisions that prevent the last variable before an inconsistency from being assigned and removes them from the set of possible decisions. This detection is done by backtracking after each conflict and selecting the last assigned variable in priority, until the constraint network can be made consistent again. Here also, the search exists in both binary and split forms but only the binary one is tested below. Variable and value ordering heuristics can be specified by the user.

Conflict ordering search

This search is presented by Gay et al. [11]. It is an extension of last conflict search, where the main difference is that the last conflict variable ordering heuristic progressively replaces the initial heuristic. In LC search, as soon as a conflict is resolved, the initial variable ordering heuristic is used again. In CO search, the initial heuristic is progressively replaced by an heuristic that selects in priority the last variable that was involved in a conflict. In addition to the initial variable ordering heuristic, it is possible to specify a value ordering heuristic.

5.5.2 Variable ordering heuristics

Two heuristics are explored in Section 5.6.

- Size: the variable with the smallest domain is selected first ;
- Min: the variable with the smallest element in its domain is selected first ; in case of ties, a preference is given to, in order, execution status, then start boundaries, then end boundaries, and finally durations.

Other heuristics were created that used information from the networks (*e.g.* most logically-constrained and most temporally-constrained). As they yielded poor performances overall, they are not included in the performance assessment.

5.5.3 Value ordering heuristics

Similarly, two heuristics are explored in Section 5.6. Unlike for variable ordering, value ordering should aim at selecting success-first values, *i.e.* values that have the largest chance to lead to a solution.

- Min: the smallest value from the domain is selected first ;
- Custom: the smallest value from the domain is selected first, except for the execution status of CTI variables, where the biggest value is selected first. This means that the search tends to first try to bind the execution status of CTI variables to 1, or *true*.

5.6 Performance

In this section, search methods will first be tested on four models, in order to determine which one is the fastest at resolving each of them. Afterwards, the best search method is used to compare the performance of each model against its reified counterpart.

5.6.1 Search methods

Each model is tested with different numbers of CTI variables. It is started multiple times, with values instantiated at random (seeded from 0 to 10), in order to compare the mean execution times of each search method. Each search method is described in Table 5.3.

Table 5.3: Search methods and their abbreviations

Branching scheme	Variable ordering heuristic	Value ordering heuristic	Abbreviation
First Fail	Size (default)	Min	FF-min
		Custom	FF-custom
Last Conflict	Size	Min	LC-size-min
		Custom	LC-size-custom
	Min	Min	LC-min-min
		Custom	LC-min-custom
Conflict Ordering	Size	Min	CO-size-min
		Custom	CO-size-custom
	Min	Min	CO-min-min
		Custom	CO-min-custom

Models are built as such:

1. Model 1 is a combination of **Alternative**, **Span**, **NoOverlap** and **Equivalence** constraints. This is an optimization problem.
2. Model 2 only uses **Span** and **NoOverlap** and is a CSP, not a COP. Indeed, finding an initial solution quickly is important, especially in the case of a Large Neighbourhood Search (LNS), where search is restarted from the initial solution but with some variables being relaxed to their initial domain.
3. Model 3 does not use CTI-specific constraints, it only constrains a quarter of the CTI variables to be executed while minimizing the sum of their durations.
4. Model 4 is a COP using **Span**, **Equivalence** and **NoOverlap** constraints.

They are available in Appendix D.

A comparison of mean solving times is available in Figures 5.9 and 5.10. While the methods yield similar results for three models out of four, **LC-size-custom** yields the best results overall, and is almost ten times more efficient than the slowest method for model 1.

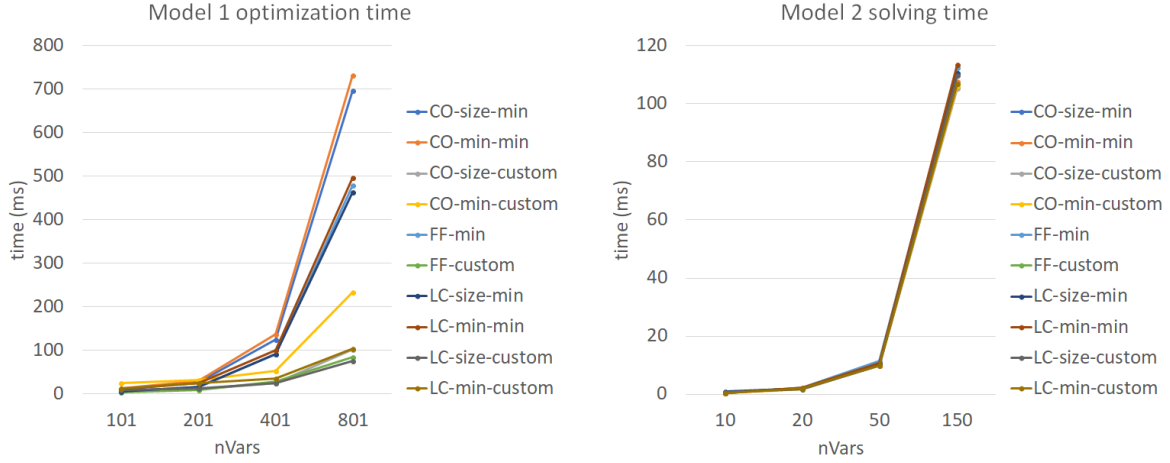


Figure 5.9: Mean solving time per search method, models 1 and 2

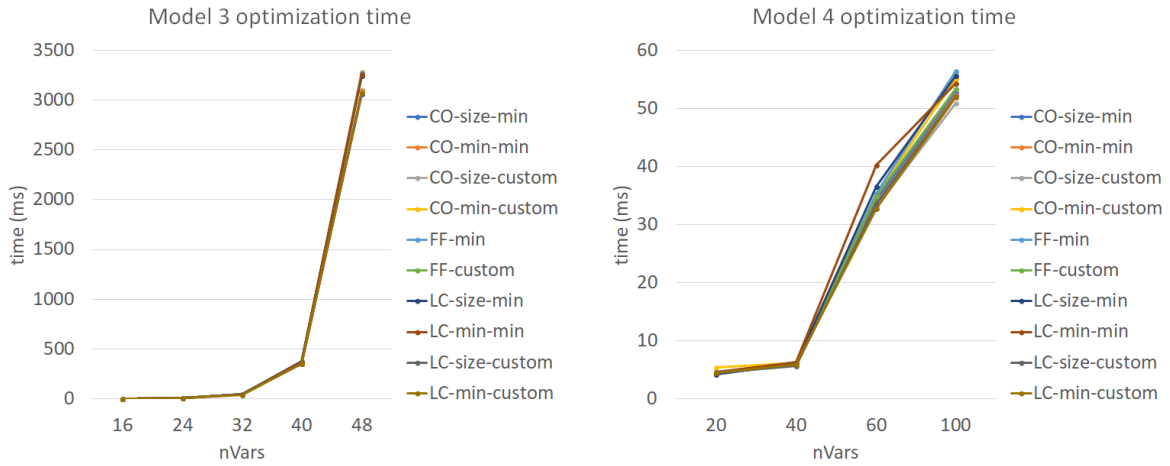


Figure 5.10: Mean solving time per search method, models 3 and 4

5.6.2 CTI against reified constraints

CTI variables do not expand the range of models that can be represented using traditional CP modeling techniques. In fact, every model using CTI variables can be modelled with traditional variables as used in most scheduling models. The conditional aspect of the variable can be expressed with reified constraints, as shown in Algorithm 6.

Algorithm 6 Reified constraint in OsaR

```

val x_i: CPBoolVar = CPBoolVar()
val assignStart: CPBoolVar = start_i ?== 5
cp.add(x_i ==> assignStart)

```

`assignStart` is a `CPBoolVar` representing the fact that $start_i$ is bound to the value 5. The third line states that, if x_i is true, *i.e.* i is executed, then $start_i$ must be equal to 5. The constraint does not hold if $x_i \neq true$.

The advantages of CTI variables therefore lie in an easier expressibility of models, automatic constraints between starts, ends and durations, and the inference of information by the logical

network which helps propagation. The four models presented above were therefore also built using reified constraints. The semantics of each pair of corresponding models is consequently the same, and seeds were used to instantiate the variables for each test randomly, but with the same values for each pair. `LC-size-custom` was used for every model, as it was the branching scheme that yielded the best results.

Results are shown in Figures 5.11 and 5.12. The models with CTI variables are clearly faster, apart for model 3. This can be explained by the fact that this model does not contain any CTI-specific constraint, and therefore does not take advantage of the logical network.

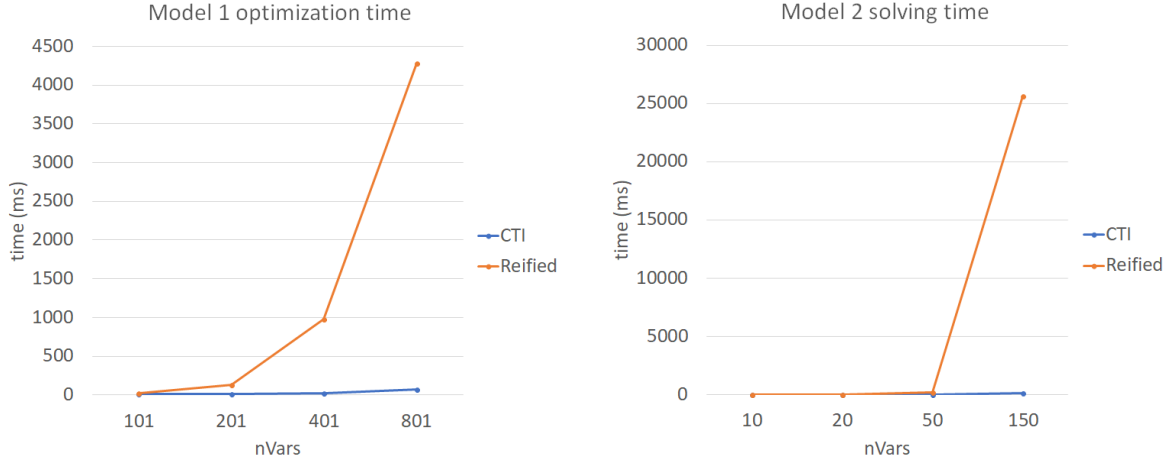


Figure 5.11: Paired comparison of mean solving times for models 1 and 2

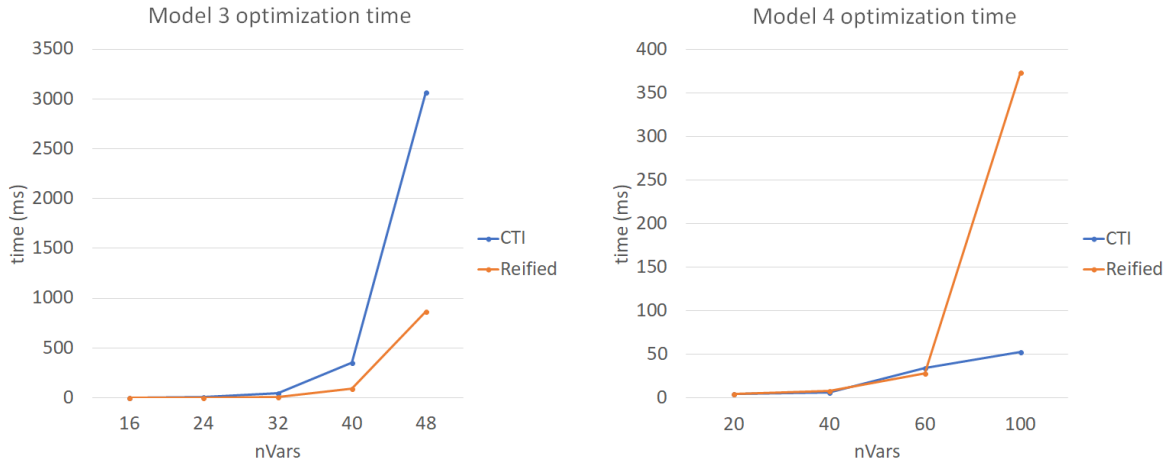


Figure 5.12: Paired comparison of mean solving times for models 3 and 4

An other interesting metric is the number of nodes that the tree search needs to explore to reach a solution for a CSP, or the optimum for a COP. The objective is of course to prune the maximum number of nodes, *i.e.* the fewer nodes, the better. A comparison for the same models in the same context is available in Table 5.4. The models using CTI variables explore fewer nodes, apart from the model 3, for the same reasons as above. In the other models, the difference is tremendous, with the majority of the paired numbers of nodes not even being in the same order of magnitude.

Table 5.4: Mean number of nodes per model and per number of variables

Model 1	# vars	101	201	401	801	
	CTI	69	52	47	44	
	Reified	1275	4601	21629	72649	
Model 2	# vars	10	20	50	100	150
	CTI	16	29	56	155	166
	Reified	50	301	2579	10914	26322
Model 3	# vars	16	24	32	40	48
	CTI	211	875	5421	39702	333761
	Reified	819	2404	6628	37266	288221
Model 4	# vars	20	40	60	100	
	CTI	166	248	731	1829	
	Reified	249	387	2101	12754	

Seeing this difference, one would expect for the speed performance difference to be even more marked. This is not the case as the CTI-type propagation is a trade off: there are far fewer nodes that are being generated than in models using reified variables, but the propagation at each node takes more time. It is however clear that the propagation slow-down is outbalanced by far by the reduction in the number of nodes that need to be explored.

5.7 A first conclusion and suggestions for further work

While the current implementation of CTI variables presents an advantage in some models, it is not always the case. The structures behind it (notably both temporal and logical networks) are heavy to maintain, and although the information they can infer helps speed-up the search in complex models, they represent overwork in simpler models. This means that there is place for further work, in order to keep the advantages of the networks while reducing the complexity and solving time on simpler models. Some interesting contributions that can be worth exploring are:

- Experiments on different data structures for both networks ;
- Incremental transitive closure computation for the logical network ;
- A detection of cases where every activity is mandatory, in order to not instantiate nor maintain the logical network.

The following chapters focus on rescheduling using models that use CTI variables. These models are available in both CP Optimizer and OscaR. A comparison of their respective performance on some examples are available in Appendix C. As the models built with CP Optimizer are significantly faster, the comparison between traditional methods and the models are assessed with the CP Optimizer models. This therefore concludes the part of this thesis focusing on OscaR.

Chapter 6

The initial model

This chapter presents the model proposed by Cappart and Schaus [6] to solve timetable rescheduling problems. It is a basis to the final model presented in the next chapter.

6.1 Signaling principles

The Belgian railways relies on the Traffic Control (TC) to schedule all of its trains. Ordinarily, the traffic is automatically regulated by a software system called Traffic Management System (TMS) based on the predefined timetables and rules.

However, in case of disturbances or disruptions, the TMS detects future conflicts and notifies the TC. The decisions related to rescheduling and rerouting of trains are then done manually by an operator. The actions to be made on the field are forwarded to the local signalling and switching towers which are responsible for the security.

The purpose of the model presented in this chapter is to help the operator to take a decision in such situation.

A layout of a fictive train station as presented in [6] can be found on Figure 6.1. A train station is composed of multiple elements:

- The **track segments** (*e.g.* $T1$) are portions of railway tracks. They are delimited by joints ($\dashv$) on the figure. Thanks to the TMS, it is possible to detect on which track segments the trains are located at any moment.
- The **signals** (*e.g.* $S1$) are used to control the traffic. They can be either set to red if the train is not allowed to go further at the moment or to green if it is safe to pass through.

Because the layouts also have movable switches, there are logical structures that also need to be considered:

- The **routes** consist of paths that trains can follow in order to reach their destination. They are defined as a sequence of track segments and signals. For instance, $R1$ is a route having $T4$ as origin and $T7$ as destination. The following elements compose the path to follow to reach the destination from the origin: $[T4, S2, T5, T2, T6]$. Note that the destination is not part of the path itself. Additionally, a route always contains a signal as second element.

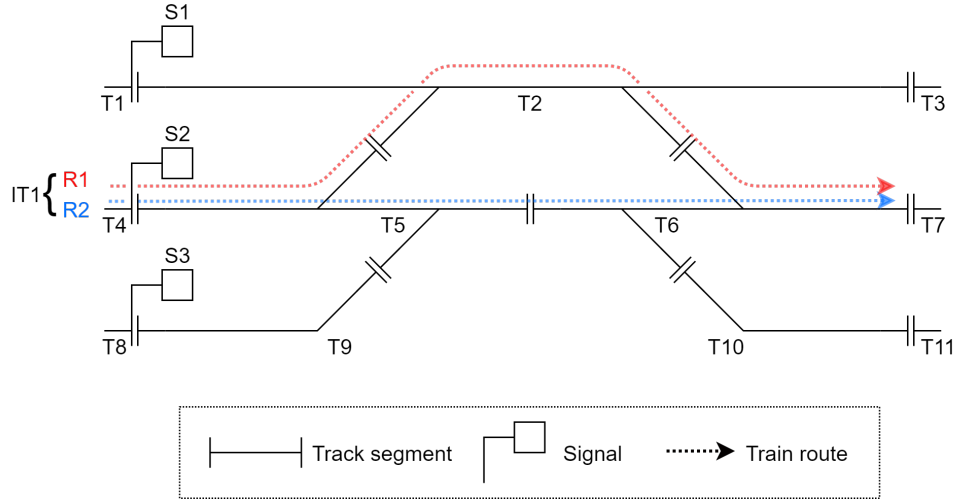


Figure 6.1: Layout of a fictive train station

This signal is always red until the TMS decides to allow the train to continue its journey, this action is called a *route activation*.

After the activation of a route, all the track segments following the signal are reserved for the train. They are released sequentially after the train passed them. For instance, once the train has reached $T2$ and is not occupying $T5$ anymore, then the latter can be released.

- The **itineraries** correspond to the non physical paths between two locations within the station. It can be constituted by one or more routes. For example, in the Figure 6.1, we can see that $R1$ and $R2$ both lead to the accomplishment of $IT1$ (from $T4$ to $T7$). Usually, a route is always preferred. However, in case of conflicts, the route causing the least amount of conflicts is to be chosen.

6.2 Elements

This section explains how the problem was modelled by Cappart and Schaus [6]. Remember that the goal here is to schedule the trains adequately to bring them to their destination. Practically, it consists of choosing for each train which route to use and when to activate it. For security purposes, the model should ensure that the elements of the train station are always used by at most one train.

6.2.1 Activities

The main components of the scheduling problems are activities. Basically, an activity A is modelled with three variables, a start date $s(A)$, a duration $d(A)$ and an end $e(A)$. The activity is considered to be fixed when the domains of its variables are reduced to singletons. There are three kinds of activities that are considered in the model:

- The **itinerary activities** define a time interval during which the train is following an itinerary. Each train has one and only one itinerary activity to accomplish. $A^{t,it}$ corresponds to the activity for itinerary it of train t .

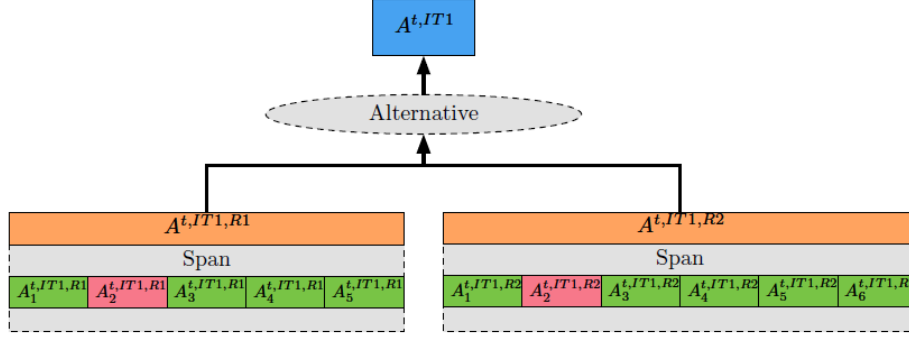


Figure 6.2: Breakdown structure of the alternative and span constraints on activities [6]

- The **route activities** define a time interval during which a particular route of an itinerary is followed by a train. $A^{t,it,r}$ stands for the route activity for route r of itinerary it related to train t .
- The **element activities** correspond to the movements of the train along the route. Such activities are associated to track segments and signals. We write $A_i^{t,it,r}$ as the i^{th} element activity for route r of itinerary it related to train t . There are as many element activities as the number of components of the route. For instance, using the example of Figure 6.1, $A_1^{t,IT1,R1}$ is an element activity corresponding to the track segment $T4$ while $A_2^{t,IT1,R1}$ applies to $S2$, etc.

A particularity of this model is that some activities are optional – they may or may not be executed. For example, if we want to accomplish $IT1$ in Figure 6.1, we could either use $R1$ or $R2$. If one of them is chosen, then the activity corresponding to the other route will not be executed. Such activities can be easily modelled using the conditional time-interval variables presented in Chapter 4.

Figure 6.2 shows how the different activities are organised in the model. $A^{t,IT1}$ is mandatory as we need to ensure the train reaches its destination, the other activities are optional. The **alternative** constraint ensures that only one route will be chosen for itinerary $IT1$. In other words, either $A^{t,IT1,R1}$ or $A^{t,IT1,R2}$ will be executed but not both. Furthermore, the start and end date of the chosen route is synchronised with the start and end dates of the itinerary activity.

The **span** constraint is used to synchronise the start of the route activity with the start of the first element activity of this route and its end with the last element activity of this route. For instance, $A^{t,IT1,R2}$ is synchronised with the start date of $A_1^{t,IT1,R2}$ and the end date of $A_6^{t,IT1,R2}$.

6.2.2 Parameters

A few parameters which characterise the trains and the track segments are considered in the model. Table 6.1 recaps the meaning of each parameter.

Table 6.1: Parameters of trains and track segments in the model

Entity	Parameter	Name	Meaning
Train	Speed	spd_t	Speed of t
	Passengers	p_t	Number of passengers of t
	Estimated Arrival Time	eat_t	When t will arrive at the station
	Earliest Start Time	est_t	Lower bound on the start time of t
	Planned Completion Time	pct_t	When t is supposed to complete its journey
	Category	c_t	Category of t
Track segment	Length	lgt_{ts}	Length of ts

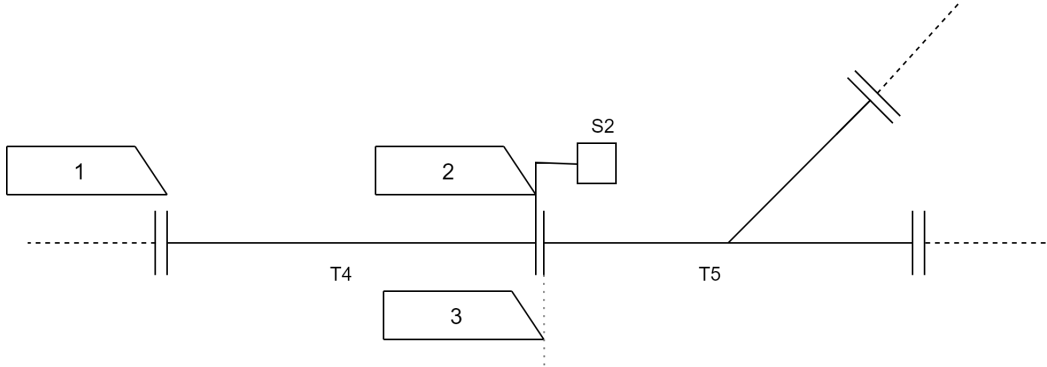


Figure 6.3: Position of the train on a station

6.2.3 Decision variables

As explained above, the goal of our model is to decide which route to execute and at what time for each train. There are a few more points to be considered in order to obtain a safe model:

- The end of an element activity must be synchronised with the start of the subsequent element activity along a route.
- An element activity can retain more than one resource. Indeed, when a train is on a track segment, it reserves the track segment as well as the next track segments on the route. For example, let us consider once again the route activity $A^{t,IT1,R1}$ depicted on Figure 6.1. The corresponding element activities are $A_1^{t,IT1,R1}$, $A_2^{t,IT1,R1}$, $A_3^{t,IT1,R1}$, $A_4^{t,IT1,R1}$ and $A_5^{t,IT1,R1}$. The resources used by these activities are $(T4, S2)$, $(S2)$, $(T5, T2, T6)$, $(T2, T6)$, $(T6)$, respectively. Note that the reservation goes no further than $S2$ for the two first activities: this represents the fact that the route activation has not occurred during these activities.

The position of the train is given by the activity it is executing at the moment and its associated element, which corresponds to the first element of its resources. For instance, remember that during $A_1^{t,IT1,R1}$, t is considered as on $T4$ while during $A_2^{t,IT1,R1}$, it is considered as in front of $S2$. This is shown on Figure 6.3. The first train shows the position of a train at the beginning of activity $A_1^{t,IT1,R1}$ whereas the second one shows its position at the end of $A_1^{t,IT1,R1}$ and the start of $A_2^{t,IT1,R1}$. The third train shows the end of the latter and the start of $A_3^{t,IT1,R1}$.

From a scheduling point of view, the problem is to assign a unique value to each element activity. The decision variables and their domains for each train t , itinerary it , route r and index i are listed below:

$$s(A_i^{t,it,r}) \begin{cases} \in [eat_t, horizon] & \text{if } t \text{ is on track segment } ts \\ \in [est_t, horizon] & \text{if } t \text{ is in front of a signal} \end{cases} \quad (6.1)$$

Equation 6.1 states that an activity cannot begin before the estimated arrival time of t . The horizon is an upper bound of the starting date of the activity which is fixed by the user.

$$d(A_i^{t,it,r}) \begin{cases} = lgt_{ts}/spd_t & \text{if } t \text{ is on track segment } ts \\ \in [0, horizon] & \text{if } t \text{ is in front of a signal} \end{cases} \quad (6.2)$$

Equation 6.2 gives us the time needed to perform an element activity. Naturally, if t is on a track segment, the duration of the element activity will be the time needed to pass through it. Otherwise, if t is located in front of a signal, then the time it will have to wait is unknown.

$$e(A_i^{t,it,r}) = s(A_i^{t,it,r}) + d(A_i^{t,it,r}) \quad (6.3)$$

Equation 6.3 is an implicit consistency constraint. It expresses the fact that the end date of an activity must be equal to the sum of its start date and its duration.

$$x(A_i^{t,it,r}) \in \{0, 1\} \quad (6.4)$$

Lastly, Equation 6.4 indicates that the activity is optional.

6.3 Constraints

This section describes the constraints that are used in the model. Let us first define T as the set of trains, IT_t as the set of possible itineraries for $t \in T$, R_{it} the set of possible routes for $it \in IT_t$, N_r as the number of element activities for a route $r \in R_{it}$ and TS as the set of track segments within the station.

6.3.1 Precedence

$$e(A_i^{t,it,r}) = s(A_{i+1}^{t,it,r}) \quad \forall t \in T, \forall it \in IT_t, \forall r \in R_{it}, \forall i \in [1, N_r) \quad (6.5)$$

This constraint (Equation 6.5) enforces the element activities to be executed in order.

6.3.2 Execution consistency

$$x(A_1^{t,it,r}) \equiv x(A_i^{t,it,r}) \quad \forall t \in T, \forall it \in IT_t, \forall r \in R_{it}, \forall i \in [1, N_r] \quad (6.6)$$

Because some routes lead to the achievement of the same itinerary, some of them are not chosen for a train. In this case none of the activities $A_i^{t,it,r}$ will be executed. Otherwise, all of them must be executed. It is expressed in Equation 6.6 - all or none of the element activities that compose a route activity are to be executed.

6.3.3 Alternative

$$\text{alternative}(A^{t,it}, \{A^{t,it,r} | r \in R_{it}\}) \quad \forall t \in T, \forall it \in IT_t \quad (6.7)$$

This constraint was introduced by Laborie and Rogerie [14] and presented in Subsection 4.2.3. It means that when the itinerary activity $A^{t,it}$ is executed, then exactly one of the route activities $A^{t,it,r}$ must be executed. Moreover, the start and end date of the itinerary and route activities must be synchronised.

Let us notice that when $A^{t,it}$ is not executed, then none of the route activities can be executed. This is why all $A^{t,it}$ are specified as mandatory in the model in order to ensure that every train reaches its destination.

6.3.4 Span

$$\text{span}(A^{t,it,r}, \{A_i^{t,it,r} | i \in [1, N_r]\}) \quad \forall t \in T, \forall it \in IT_t, \forall r \in R_{it} \quad (6.8)$$

Like **alternative**, this constraint was introduced in [14] but also presented in Subsection 4.2.3. It states that an executed route activity $A^{t,it,r}$ must span over some of the executed element activities that compose the route. It also ensures that the start of the route activity is synchronised with the start of the first underlying element activity as well as the end of the route activity is synchronised with the end of the last element activity.

Using this constraint, it is easy to model the fact that the duration of a route activity is equal to the time taken to cross all of its components.

6.3.5 noOverlap - unary resource

An important part of the model is the security aspect. Indeed, it must be impossible for a train to move onto or reserve a track segment that is currently used by another one.

Let us state that ACT_{ts} is the set of all activities using the track segment ts :

$$\text{noOverlap}(A | A \in ACT_{ts}) \quad \forall ts \in TS \quad (6.9)$$

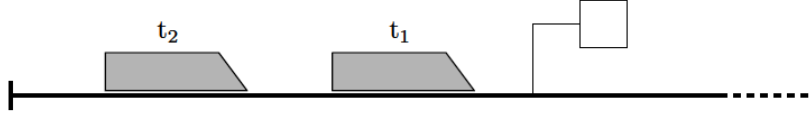


Figure 6.4: Two trains waiting on the same track segment [6]

This constraint was introduced in [15]. Its semantic is presented in Subsection 4.2.2. It states that the set of all executed activities in ACT_{ts} must form a chain of non-overlapping intervals *i.e.* any executed activity in the chain must finish before the start of the next executed activity.

6.3.6 Train Ordering consistency

This constraint states that trains cannot overtake other trains if they are located on the same track segment. Such a situation is illustrated on Figure 6.4. It is clear that train t_2 may not start its activities before t_1 , even if t_2 has a higher priority.

To model this, let us state $TSB \subset TS$ as the set of all first track segments, N_{tsb} as the number of trains beginning their itinerary on track segment tsb , and AB_i^{tsb} as the sequence of first activities of trains t_i beginning on tsb with $i \in [1, N_{tsb}]$, ordered by estimated arrival time of trains t_i . Then, the train ordering consistency constraint can be modelled as shown in Equation 6.10.

$$s(AB_{i-1}^{tsb}) < s(AB_i^{tsb}) \quad \forall tsb \in TSB, \forall i \in]1, N_{tsb}] \quad (6.10)$$

6.4 Objective functions

This section presents the objective functions used on homogeneous (when there is only one category of trains) and heterogeneous (when we have multiple categories of trains) models.

6.4.1 Homogeneous model

The most frequent criterion used as objective function in this case is the sum of train delays. To express this, let us first define jct_t as the *journey completion time of train t*. It is equal to the end date of the last element activity of train t . The delay d_t of train t can thus be expressed as:

$$d_t = \max(0, jct_t - pct_t) \quad (6.11)$$

The $\max$ function is used to nullify the cases where trains reach their destinations in advance. From Equation 6.11, we can state the objective function of the homogeneous model as:

$$\min \left(\sum_{t \in T} d_t \right) \quad (6.12)$$

6.4.2 Heterogeneous model

However, in reality, the operators must consider different categories of trains introduced in Table 6.1 having different priorities. The four considered categories of trains are listed below in decreasing order of priority:

1. Maintenance trains ($C1$).
2. Passenger trains with correspondence ($C2$).
3. Passenger trains without correspondence ($C3$).
4. Freight trains ($C4$).

Moreover, there is now the possibility to take the number of passengers inside the trains into consideration. The objective function becomes then threefold:

- Schedule trains according to their priority.
- Minimise the overall delay.
- Maximise the overall passenger satisfaction which decreases if their train is delayed.

We end up with the following objective function:

$$\text{lex_min}\left(\sum_{t \in C1} d_t, \sum_{t \in C2} p_t \times d_t, \sum_{t \in C3} p_t \times d_t, \sum_{t \in C4} d_t\right) \quad (6.13)$$

The Equation 6.13 gives a lexicographical ordering of trains according to their categories and priorities. Remember that p_t stands for the number of passengers inside train t . Thus, we are taking into account the number of passengers in the minimisation of the delay for categories $C2$ and $C3$. Finally, the objective is to minimise the expression while respecting the lexicographical ordering.

6.5 Experiments and results

To sum up this chapter, the model has been experimented in [6] on a subset of a Belgian train station (Courtrai). The main results are as follows:

- The CP model was compared against classical scheduling approaches, namely first come first served (FCFS), highest delay first served (HDFS) and highest priority first served (HPFS).

In the end, the homogeneous model gave an average improvement ratio of 20% in terms of objective value compared to HDFS and FCFS with a decision time of 3 minutes for instances between 5 and 30 trains. It gave better or equal results than the classical approaches in more than 99% of the cases. The optimum was reached more than 75% of the time when considering instances of 10 trains or less.

The heterogeneous model also gave better results in almost all the cases (99% of the instances) when compared to HPFS with a decision time of 3 minutes and instances between 5 and 30 trains.

- The solution is scalable - it allows to obtain competitive results in less than three minutes for instances up to thirty trains.
- The model is also nice in critical situations. It was tested with a decision time of 10 seconds against HDFS and FCFS and still gave better or equal results in 99% of the cases.

6.6 Limitations of the initial model

Even though the model presented above shows good performances and a nice way to deal with such scheduling problem, it is still limited in some ways. This section presents some of the limitations we identified in the initial model. The solutions which deal with these limitations are presented in the next chapter.

6.6.1 Length of the trains

Even though this parameter was part of the model presented by Rodriguez [20], the lengths of the trains are not considered in the model. This parameter is useful to consider as it makes the model a closer simulation of the reality. Indeed, in practice, the trains have different lengths and thus occupy the elements of the station differently.

6.6.2 Platforms

Another aspect that is not considered in the initial model is the presence of platforms as part of the train stations and the fact that some trains have to stop at these platforms.

6.6.3 Considering larger areas and multiple stations

As it was identified in [4] and in Chapter 2, the optimisation models which deals with rescheduling trains in a microscopic view have only been tested on rather small railway networks. Hence, an interesting experiment would be to consider the model on larger areas and multiple train stations. This is also suggested in [6].

6.6.4 Encountering signals after route activation

The model assumes that routes are only composed of track segments after the start signal. However, in reality, this might not be true. For instance, a train might encounter a signal at a platform and thus might need to stop after its route has been activated.

Chapter 7

The new model

This chapter introduces our new model. It presents the extensions included in the model presented in the last chapter following the limitations identified in Section 6.6.

Concretely, the new model deals with trains of variable lengths, platforms as part of the stations, large areas *i.e.* multiple stations, and elements other than track segment after the route activation.

7.1 Length of the trains

As explained in the last chapter, the length of the trains is an interesting parameter to consider. This section first explains the problem in more details, then discusses which parts of the initial model need to be changed and finally presents some solutions.

7.1.1 The problem

When considering trains of variable length, the duration of the elements activities also depends on this parameter. Indeed, the rear carriages of the train can still occupy the element linked to the activity even though it has ended.

Figure 7.1 shows the possible positions of a train on the track segment $TS2$ at some different time of the associated element activity. The first position of the train corresponds to the start of the element activity, when the train starts to use the track segment. The second position shows the position of the train at the end of the element activity in the initial model. The third position shows the position of the train when it has completely freed the track segment *i.e.* when its rear carriages are not anymore on the track segment.

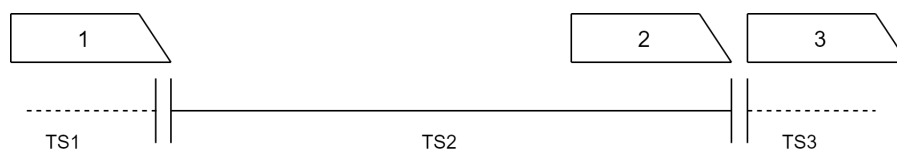


Figure 7.1: Possible positions of a train on a track segment when considering its length

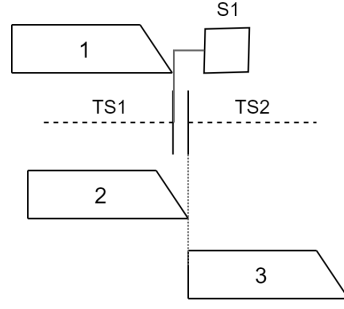


Figure 7.2: Possible positions of a train in front of a signal when considering its length

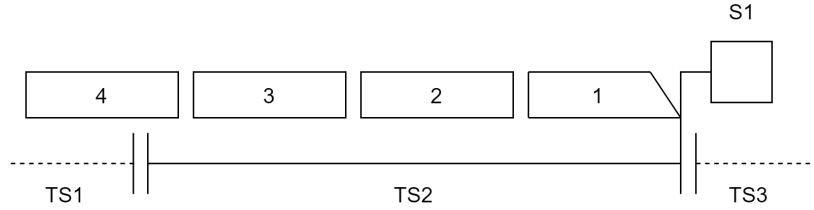


Figure 7.3: Train waiting at signal having its rear carriages occupying preceding track segments

Likewise, Figure 7.2 shows the possible positions of a train in front of the signal $S1$ at some different time of its corresponding element activity. As in the previous figure, the first position shows the start of the activity, the second one shows its end and the third one shows when the train effectively liberated the signal with all its carriages.

As only one train is allowed per element, the liberation of the element should happen only when the train reaches position 3 in both cases *i.e.* when all its carriages freed the element.

Finally, the process of making a train wait at a red signal becomes more complicated when adding the length of the trains as parameter of the model. If a train is stopped in front of a signal, it is possible for its rear carriages to occupy one or more (depending on both the size of the train and the preceding elements) elements preceding the signal. For instance, on Figure 7.3, the train occupies $TS1$ and $TS2$ while being stopped in front of $S1$.

7.1.2 Parts of the model to be modified

As explained above, the initial model does not take the length of the cars of the train into account in the computation of the duration of the element activities.

In order to ensure that all the parts of the train are not anymore on a track segment, the duration of the elements activities expressed in Equation 6.2 have to change when taking this parameter into account.

Concretely, the model needs to create activities such that it can determine when the train is at the three possible positions shown on Figure 7.1 and Figure 7.2 at all time. The initial model only keeps track of the two first positions, we thus need to find a way to handle the third one as well. The usefulness of keeping track of these position is listed below:

- The first position is useful to know when the element activity starts.

- The second position is needed to know when to start the activity related to the next element ($TS3$ in Figure 7.1, $TS2$ in Figure 7.2). This refers to the precedence constraint presented in Subsection 6.3.1.
- The third position indicates when to release the element. This implies the `noOverlap` constraint presented in Subsection 6.3.5. Remark that the set ACT_{ts} presented in the same subsection can be extended to handle signals and track segments instead of the latter only. This would allow to ensure that the rear carriages have liberated a signal before freeing it.

Finally, Figure 7.3 shows that trains may occupy preceding elements when stopped at signals. To fix this issue, the model requires to create activities such that when the situation occurs, a train also reserves the preceding elements, depending on both the size of the train and the preceding elements.

To sum up, the changes to perform to the initial model are listed below:

- Create new element activities to handle the rear carriages of the trains (track position 3 in Figure 7.1 and 7.2).
- Create new element activities to reserve the element preceding a signal when the train is stopped in front of it.
- The duration of these activities are different from the one presented in the initial model expressed in Equation 6.2.
- The newly created activities need to be used in the `noOverlap` constraint (Equation 6.9) to ensure only one train per element at all time. The ACT_{ts} set can be extended to include signals.
- All of this has to be done while keeping the initial elements activities to keep track of the two first positions of the train presented in Figure 7.1 and 7.2.

7.1.3 Solutions

This subsection presents our solutions to the problem discussed above.

As first step towards a solution, the set ACT_{ts} presented in Subsection 6.3.5 is extended to handle the signals as well. Let us define E as the set of all the elements (track segments and signals) in the train station. Let us then state that ACT_{elem} is the set of all activities using the element $elem \in E$.

There are multiple ways to cope with the rest of the problem. Before jumping into the solutions, let us first introduce the concepts of head activity and tail activity.

- A **head activity** is an activity that tracks the foremost part of the train. Basically, it is a new term for an element activity as understood in the initial model. It is introduced to differentiate them with the tail activities.

- A **tail activity** is an activity that tracks the rear end of the train. The introduction of such activities allows to track all the positions discussed above.

There are two ways to create and synchronize the tail activities based on the initial model. They both can be used to resolve the problem.

End synchronized tail activities

A first possibility is to create for each element activity a new tail activity. The constraints related to these new decision variables are listed below:

- The new tail activity lasts the time needed for a train to cross a distance equals to its size (lgt_t/spd_t , with lgt_t being the length of train t).
- Its starting date is synchronised with the end of its corresponding head activity - this is why it is called end synchronized.
- It has the same execution status as its corresponding head activity.
- It is attached to the first resource of its corresponding head activity only.

Remember the terms defined in Section 6.3: T was defined as the set of trains, IT_t as the set of possible itineraries for $t \in T$, R_{it} the set of possible routes for $it \in IT_t$, N_r , and as the number of element activities for a route $r \in R_{it}$. Let us now define $A_i^{head,t,it,r}$ as the i^{th} head activity of train t related to itinerary $it \in IT_t$ and route $r \in R_{it}$ and $A_i^{tail,t,it,r}$ as the corresponding tail activity. Formally, the following constraints are added to the model for each train t , itinerary $it \in IT_t$, route $r \in R_{it}$, $i \in [1, N_r]$:

$$d(A_i^{tail,t,it,r}) = lgt_t/spd_t \quad (7.1)$$

This constraint sets the duration of the tail activity to lgt_t/spd_t , the time needed for a train to cross a distance equals to its size. In Figure 7.1 and 7.2, this corresponds to the time needed for the train to go from position 2 to position 3.

$$s(A_i^{tail,t,it,r}) = e(A_i^{head,t,it,r}) \quad (7.2)$$

The constraint 7.2 ensures that the tail activity is beginning at the end of the head activity. It starts when the train reaches position 2 in the figures mentionned earlier.

$$x(A_i^{tail,t,it,r}) \equiv x(A_i^{head,t,it,r}) \quad (7.3)$$

This constraint enforces both or none of the activities to be executed.

$$e(A_i^{tail,t,it,r}) = s(A_i^{tail,t,it,r}) + d(A_i^{tail,t,it,r}) \quad (7.4)$$

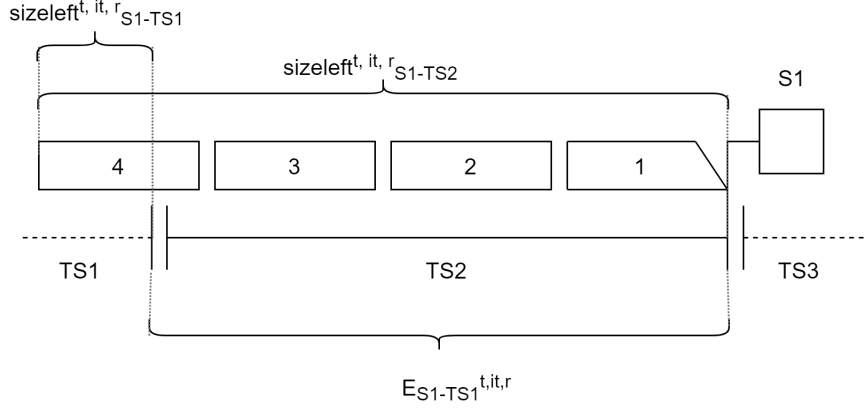


Figure 7.4: Figure 7.3 updated with sizeLeft and the $E_{s-e}^{t,it,r}$ set

The above constraint is an implicit consistency constraint as Equation 6.3. It expresses the fact that the end of the tail activity must be equal to the sum of its start date and its duration.

There is a direct correspondence between the positions shown in Figures 7.1 and 7.2, and the start and end dates of the head and tail activities. The first one shows the position of the train when it starts the head activity ($A_i^{head,t,it,r}$) linked to the resources ($TS2, TS3, \dots$) in the first figure and to ($S1$) in the second one. The second one points to when the train is supposed to be at the end of its head activity and at the start of its tail activity ($A_i^{tail,t,it,r}$). It also allows to use the precedence constraint presented in Equation 6.5 to order the activities: it marks the start of the subsequent head activity ($A_{i+1}^{head,t,it,r}$). The last position corresponds to the end of the tail activity, when it is supposed to release $TS2$ or $S1$.

Using the newly created activities, it is possible to ensure that the train reaches position 3 before releasing the element. This is done implicitly because both the head and tail activities are in the set ACT_{elem} presented above. Remember that the newly created tail activities are only linked to the first resource of the corresponding head activity. This means that they only constrain the element the train is supposed to be on or in front of during the head activity.

Taking the example of Figure 7.1 again, the head activity linked to resources ($TS2, TS3, \dots$) is by definition in $ACT_{TS2}, ACT_{TS3}, \dots$. Whereas the corresponding tail activity is only in ACT_{TS2} .

Then, the **noOverlap** constraint presented in Equation 6.9 is modified to be applied on the set ACT_{elem} instead as such:

$$\text{noOverlap}(A|A \in ACT_{elem}) \quad \forall elem \in E \quad (7.5)$$

Finally, there is still the case where a train is stopped in front of a signal having its rear carriages on the preceding elements to consider. Let us update Figure 7.3 with Figure 7.4. This new figure shows useful information to understand the new concepts introduced below.

The first thing to do to handle such cases is to determine which elements are occupied by the train t when it is using route r of itinerary it and it is stopped at the signal s . In Figure 7.4, there are two such elements: $TS1$ and $TS2$. Let us call this set of elements $E_s^{t,it,r}$.

When the head of the train is in front of the signal s , for each element $e \in E_s^{t,it,r}$, it is possible to calculate the size left of the train going from the end of the element up to the rear end of the train. Let us write $E_{s-e}^{t,it,r}$ as the set of previous elements between s and e for train t on the route r of itinerary it . For instance, in Figure 7.4, $E_{S1-TS2}^{t,it,r} = \emptyset$ (there is no element between $S1$ and $TS2$), whereas $E_{S1-TS1}^{t,it,r} = TS2$ because $TS2$ is between $S1$ and $TS1$. The size left for train t , route r of itinerary it , signal s and element e is calculated by the following equation:

$$sizeLeft_{s-e}^{t,it,r} = lgt_t - \sum_{elem \in E_{s-e}^{t,it,r}} lgt_{elem} \quad (7.6)$$

Taking the example of Figure 7.4, assuming that $lgt_t = 120$ and $lgt_{TS2} = 100$. We have $sizeLeft_{S1-TS2}^{t,it,r} = 120$ and $sizeLeft_{S1-TS1}^{t,it,r} = 20$.

For each element $e \in E_s^{t,it,r}$, it must be impossible for two different trains to reserve or use this element simultaneously. This issue is solved by adding an extra tail activity synchronized with the end of the tail activity of the corresponding element for each $e \in E_s^{t,it,r}$. Let us call $S^{it,r}$ the set of signals along route r of itinerary it , $A_{s-e}^{tail,t,it,r}$ the tail activity of train t following itinerary it and route r being stopped at signal s with preceding element $e \in E_s^{t,it,r}$ and $A_e^{tail,t,it,r}$ the tail activity of train t following itinerary it and route r corresponding to when it passes on or is stopped in front of element e . The following constraints are then added to the model for each train t , itinerary $it \in IT_t$, route $r \in R_{it}$, signal $s \in S^{it,r}$ and element $e \in E_s^{t,it,r}$:

$$s(A_{s-e}^{tail,t,it,r}) = e(A_e^{tail,t,it,r}) \quad (7.7)$$

The constraint signifies that the new tail activity created from the signal ($A_{s-e}^{tail,t,it,r}$) is end synchronized with the tail activity that was created for the element ($A_e^{tail,t,it,r}$).

The end of the new activity, with $A_s^{head,t,it,r}$ being the head activity of train t corresponding to signal s is set to :

$$e(A_{s-e}^{tail,t,it,r}) = e(A_s^{head,t,it,r}) + \frac{sizeLeft_{s-e}^{t,it,r}}{spd_t} \quad (7.8)$$

The equation 7.8 states that the end of the tail activity $A_{s-e}^{tail,t,it,r}$ must happen $sizeLeft_{s-e}^{t,it,r} / spd_t$ units of time after the end of its corresponding head activity $A_s^{head,t,it,r}$. This means that when the train is allowed to pass through the signal, it will still occupy the element e for $sizeLeft_{s-e}^{t,it,r} / spd_t$ units of time, that is the time needed for the tail to clear e .

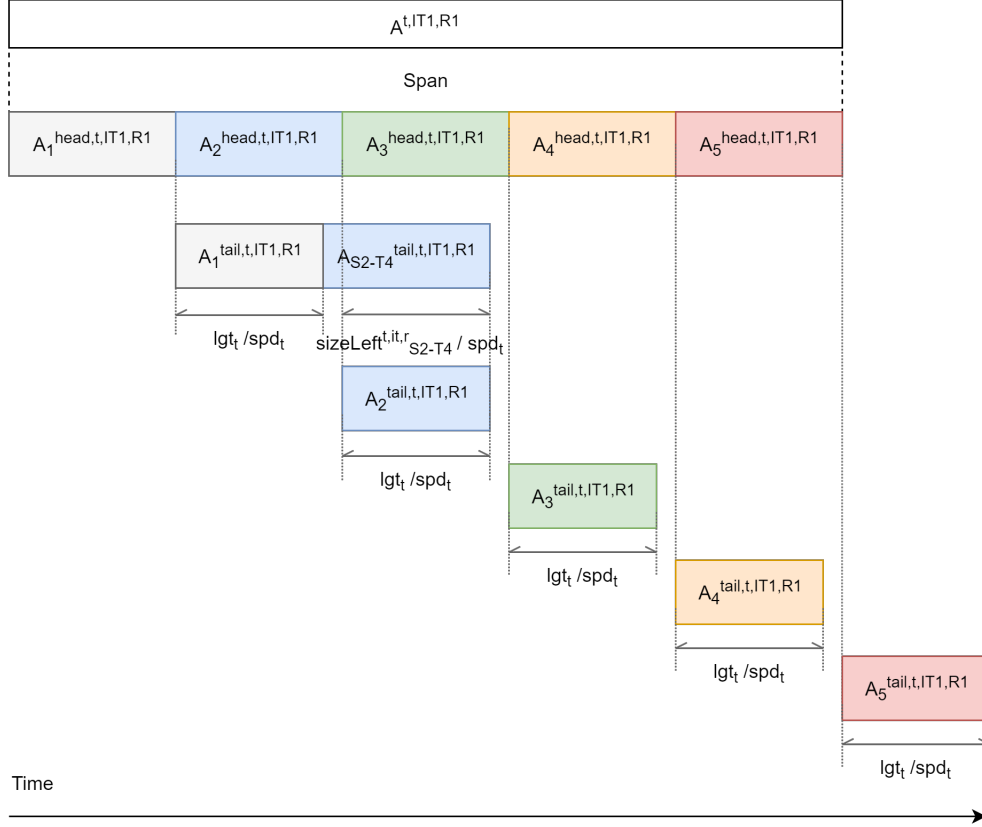


Figure 7.5: Part of the breakdown structure when adding the end synchronized tail activities to the model

And, as before, set the execution states to be the same:

$$x(A_{s-e}^{tail,t,it,r}) \equiv x(A_s^{head,t,it,r}) \quad (7.9)$$

The following consistency constraint is added as well:

$$d(A_{s-e}^{tail,t,it,r}) = e(A_{s-e}^{tail,t,it,r}) - s(A_{s_e}^{tail,t,it,r}) \quad (7.10)$$

The above constraint states that the duration of the newly created activity is equal to the difference of its end date and its start date.

The last thing to do is to add $A_{s-e}^{tail,t,it,r}$ to ACT_e and let the unary resource (**noOverlap**) constraint unchanged from Equation 7.5.

To finish this subsection, let us use the example presented in Figure 6.1. Figure 7.5 represents a part of the breakdown structure presented in Figure 6.2 with the addition of the new end synchronized activities. The colours are used to show from which head activities are created the tail activities. This structure shows how the reservation of the elements is extended by the addition of the end synchronized tail activities. Remark that the size of the tail activities can be

Table 7.1: Correspondence of elements and resources per activity with end synchronized tail activities

Activity	Created from	Ressources
$A_1^{head,t,IT1,R1}$	$T4$	$(T4, S2)$
$A_1^{tail,t,IT1,R1}$	$T4$	$(T4)$
$A_2^{head,t,IT1,R1}$	$S2$	$(S2)$
$A_{S2-T4}^{tail,t,IT1,R1}$	$S2$	$(T4)$
$A_2^{tail,t,IT1,R1}$	$S2$	$(S2)$
$A_3^{head,t,IT1,R1}$	$T5$	$(T5, T2, T6)$
$A_3^{tail,t,IT1,R1}$	$T5$	$(T5)$
$A_4^{head,t,IT1,R1}$	$T2$	$(T2, T6)$
$A_4^{tail,t,IT1,R1}$	$T2$	$(T2)$
$A_5^{head,t,IT1,R1}$	$T6$	$(T6)$
$A_5^{tail,t,IT1,R1}$	$T6$	$(T6)$

longer or shorter as it depends on the length of the train. In the figure, the train is assumed to be shorter than the track segments as the tail activities are shorter than the head activities.

Table 7.1 lists all the activities shown in Figure 7.5 along with their associated elements and resources. Remember that the activities which share at least one resource are constrained by the `noOverlap` constraint (Equation 7.5).

Start synchronized tail activity

The second way is again to create a new tail activity per element activity. The changes related to the addition of these new activities are listed below:

- The newly created activities have a duration that is lgt_t/spd_t longer than the duration of the head activity.
- These new variables have their starting dates synchronized with the starting date of the corresponding head activity - they are start synchronized.
- The execution status of head and tail activities are set to be the same.
- The `noOverlap` constraint is modified again. The newly created activities can be attached to all or only the first resource of the corresponding head activity.

This gives us the following constraints to add to the model for each train t , itinerary $it \in IT_t$, route $r \in R_{it}$, $i \in [1, N_r]$:

$$d(A_i^{tail,t,it,r}) = d(A_i^{head,t,it,r}) + \frac{lgt_t}{spd_t} \quad (7.11)$$

Remember that the term lgt_t/spd_t corresponds to the time needed to cover a distance equal to lgt_t for a train driving at spd_t . The Equation 7.11 states that the duration of an element activity lasts the time needed for the train to cross the element and to ensure that none of its

carriage is still using the element. This corresponds to the time needed to go from position 1 to position 3 in Figure 7.1 and 7.2.

$$s(A_i^{tail,t,it,r}) = s(A_i^{head,t,it,r}) \quad (7.12)$$

This constraint synchronizes the start date of both head and tail activities.

As in the previous variant, the use of constraints 7.3 and 7.4 ensures that the head and tail activities have the same execution status and that the end date of the activity is equal to the sum of its start date and its duration.

The same positions as shown in the example of Figure 7.1 and 7.2 are tracked with this variant. Only the position of the starting date of the tail activity changes: the first position is the start of both head ($A_i^{head,t,it,r}$) and tail ($A_i^{tail,t,it,r}$) activities, the second position marks the end of the head activity and the third position shows the end of the tail activity.

Here again, keeping track of the head activities allows to use the precedence constraint between head activities as in the original model while tracking the tail activities allows to know where the rear ends of the trains are. However, in this variant, the **noOverlap** constraint (Equation 7.5) has to be modified.

Because the head and tail activities of the train always overlap in this case, the addition of the tail activity to ACT_{elem} requires more work than before. A fix is to only add the **noOverlap** constraint on two head or tail activities if they belong to a different train. For this, ACT_{elem} is modified to store pairs containing the id of the train and the activity instead of the latter only. Then, the Equation 6.9 is redefined as:

$$\begin{aligned} \text{noOverlap}(\{A_1, A_2\}) \quad \forall elem \in E \\ \forall \{A_1, A_2\} \mid (id_{t1}, A_1) \in ACT_{elem}, (id_{t2}, A_2) \in ACT_{elem} \wedge id_{t1} < id_{t2} \end{aligned} \quad (7.13)$$

The $id_{t1} < id_{t2}$ part of equation is there to gain time by avoiding to add symmetrical **noOverlap** constraints. Note that we consider the activities two by two to limit the explosion of the number of **noOverlap** constraints. Because there are less combinations of size two than combinations of size $n > 2$ i.e. $C_k^n > C_k^2 \quad \forall n > 2 \quad \forall k > 2$, there are less **noOverlap** constraints to add if the activities are taken two by two.

The only thing left to deal with is the situation where a train is stopped in front of a signal having its carriages on the previous elements of the path. The solution is based on the same reasoning as presented above with the end synchronized activities, but applied to the start synchronized activities in this case. For each element $e \in E_s^{t,it,r}$, the model creates a new start synchronized tail activity $A_{s-e}^{tail,t,it,r}$ synchronized with the start of the head activity $A_s^{head,t,it,r}$.

The following constraints are then added to the model for each train t , itinerary $it \in IT_t$, route $r \in R_{it}$, signal $s \in S^{it,r}$ and element $e \in E_s^{t,it,r}$:

$$s(A_{s-e}^{tail,t,it,r}) = s(A_s^{head,t,it,r}) \quad (7.14)$$

This constraint specifies that the newly created tail activities are start synchronized with their respective head activities.

Then, the duration of the new activity is set to:

$$d(A_{s-e}^{tail,t,it,r}) = d(A_s^{head,t,it,r}) + \frac{sizeLeft_{s-e}^{t,it,r}}{spd_t} \quad (7.15)$$

This constraint means that the duration of $A_{s-e}^{tail,t,it,r}$ will include the time needed for the tail to clear the element after the signal let the train pass through. This is why it is $sizeLeft_{s-e}^{t,it,r}/spd_t$ longer than $d(A_s^{head,t,it,r})$.

The constraints 7.4 and 7.9 are used here again to enforce the consistency of the end date of the activity as well as the execution states of the head and tail activities.

Finally, in this case, the problem with the unary resource constraint **noOverlap** arises as before. The fix presented earlier (in constraint 7.13) also works here.

As for the end synchronized activities, Figure 7.6 illustrates the changed part of the breakdown structure when adding the start synchronized activities to the model. It shows when these new activities begin and for how long they last as well as the overlapping areas.

The resources used by these activities are listed in Table 7.2. With this variant, it is possible to use the same resources for the tail activities as with the end synchronized tail activities but it is also possible to use the same resources for the head and tail activities, because of the change in the **noOverlap** constraint.

Final word

The differences between the two variants can be identified when comparing the two breakdown structures as well as the tables which list the resources.

The two solutions presented in this section were tested in our model. The end synchronized tail activities give better performances overall. This is due to the fact that the start synchronized tail activities add **noOverlap** constraints two by two whereas the end synchronized activities add only one of those per element of the station. This makes the model heavier in terms of constraints and leads to worse propagation.

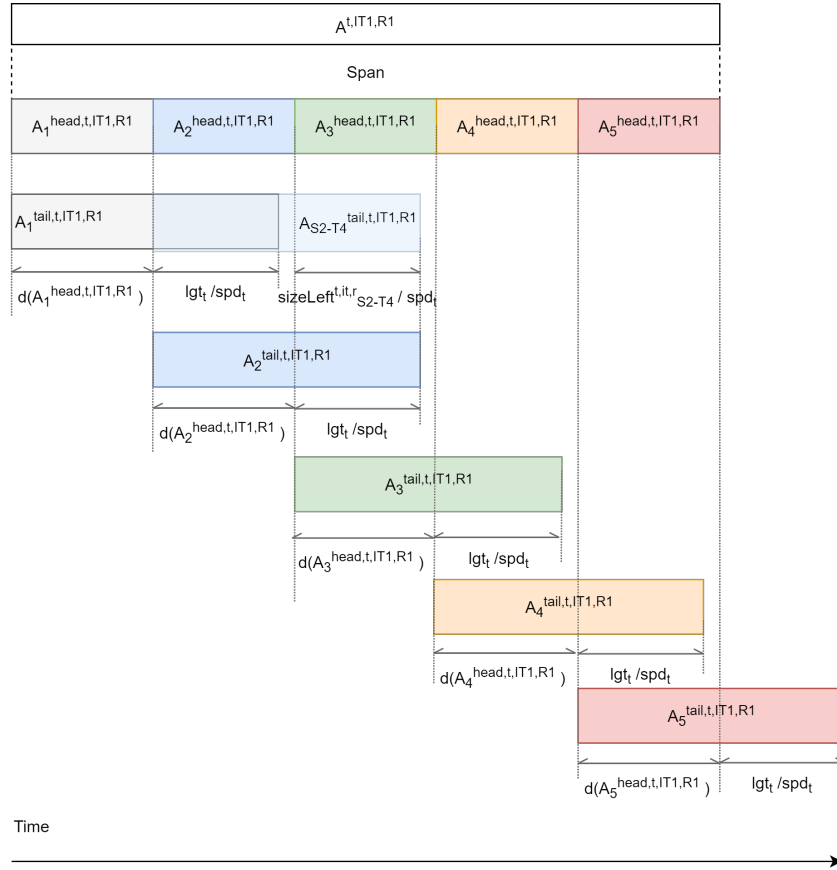


Figure 7.6: Part of the breakdown structure when adding the start synchronized tail activities to the model

Table 7.2: Correspondence of elements and resources per activity with end synchronized tail activities

Activity	Created from	Ressources
$A_1^{\text{head},t,IT1,R1}$	$T4$	$(T4, S2)$
$A_1^{\text{tail},t,IT1,R1}$	$T4$	$(T4)$ or $(T4, S2)$
$A_2^{\text{head},t,IT1,R1}$	$S2$	$(S2)$
$A_{S2-T4}^{\text{tail},t,IT1,R1}$	$S2$	$(T4)$ or $(T4, S2)$
$A_2^{\text{tail},t,IT1,R1}$	$S2$	$(S2)$
$A_3^{\text{head},t,IT1,R1}$	$T5$	$(T5, T2, T6)$
$A_3^{\text{tail},t,IT1,R1}$	$T5$	$(T5)$ or $(T5, T2, T6)$
$A_4^{\text{head},t,IT1,R1}$	$T2$	$(T2, T6)$
$A_4^{\text{tail},t,IT1,R1}$	$T2$	$(T2)$ or $(T2, T6)$
$A_5^{\text{head},t,IT1,R1}$	$T6$	$(T6)$
$A_5^{\text{tail},t,IT1,R1}$	$T6$	$(T6)$

7.2 Platforms

Subsection 6.6.2 showed that the initial model does not include the concept of platform. This section explains the problem, then discusses which parts of the initial model have to be changed and finally how our model proceeds to handle such elements.

7.2.1 The problem

Some trains are scheduled to stop at a given platform for a minimum amount of time when executing their itineraries. Thus, the model should include for every train itinerary information to specify:

- Whether the train is supposed to stop at a platform along its itinerary.
- Which platform is scheduled to receive the train.
- What is the minimum time planned for the stop.

Because it might be profitable to reroute some trains, it is possible that a train stops at a different platform from the one that was planned. While profitable for the delays in simulation, there are other costs of doing such thing in reality:

- The passengers might be confused about where to go if they arrive at a different platform from what was scheduled.
- The passengers who have a connecting train to take might take more time to reach it. Indeed, usually, the connections are scheduled to happen on adjacent platforms to reduce the connection's travelling time of the passengers by avoiding them to use the stairs for example.

7.2.2 Parts of the model to be modified

The information related to the platforms can be specified as parameters of the trains. The Table 6.1 can be modified to include these parameters. Also, because a train can be stopped at a platform for a minimum amount of time, the duration of the activities related to platforms will be different from the others. Thus, the duration of an activity expressed in Equation 6.2 needs to change.

Regarding the second part of the problem, the objective functions of the heterogeneous (Expression 6.13) and homogeneous (Expression 6.12) model can be modified to include an additional cost of rerouting a train to a platform that is not in the initial schedule.

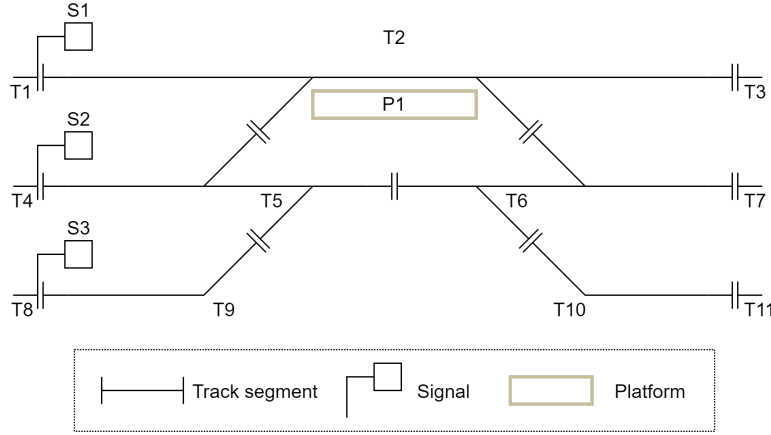


Figure 7.7: Example of a toy station with a platform

7.2.3 Solutions

Figure 7.7 shows an example of a toy station having platforms as part of it.

It is possible to consider the platforms as a special kind of signals. Indeed, the trains may or may not stop for some time when encountering such element. Two parameters are added to the train:

- ssa_t (should stop at), the name of the platform where the train t is supposed to stop. Its value is $\perp$ if the train is not supposed to stop, the name of the platform otherwise.
- mst_t (minimum stopping time), the minimum amount of time the train t is supposed to stay at the platform. It is a positive integer and its value is 0 if the train does not have to stop.

The main difference between a platform and a signal is the duration of its corresponding activity. A platform activity may have a minimum duration greater than zero (if the train must stop there) or it may be forced to be zero (in case the train does not stop at the platform). The duration equation presented in constraint 6.2 is modified as such:

$$d(A_i^{t,it,r}) \begin{cases} = lgt_{ts}/spd_t & \text{if } t \text{ is on track segment } ts \\ \in [0, horizon] & \text{if } t \text{ is in front of a signal} \\ \in [mst_t, horizon] & \text{if } t \text{ is in front of platform and } ssa_t \neq \perp \\ = 0 & \text{if } t \text{ is in front of a platform and } ssa_t = \perp \end{cases} \quad (7.16)$$

The third line of the equation corresponds to the case where the train has to stop at a platform while the last line corresponds to a train that does not need to stop. The value is set to zero because trains are unlikely to stop at a platform if they are not intended to do so.

The rest of the constraints for the platforms are the same as the signals.

The second part of the problem is solved by adding a cost to the objective function for each train that is not stopping where it is supposed to. Practically, the cost of platform change for train t (ccp_t) is calculated as follows, with $R_{diff}^{t,it}$ being the set of routes for train t and itinerary it not containing the platform ssa_t and $cost$ a constant value fixed by the user:

$$ccp_t = \sum_{r \in R_{diff}^{t,it}} x(A^{t,it,r}) \times cost \quad (7.17)$$

It is then possible to include the cost ccp_t to the objective functions of the model. The homogeneous objective function 6.12 becomes:

$$\min \left(\sum_{t \in T} d_t + ccp_t \right) \quad (7.18)$$

And the heterogeneous objective function 6.13 becomes:

$$\text{lex_min} \left(\sum_{t \in C1} d_t, \sum_{t \in C2} p_t \times d_t, \sum_{t \in C3} p_t \times d_t, \sum_{t \in C4} d_t, \sum_{t \in T} ccp_t \right) \quad (7.19)$$

The choice of order in Equation 7.19 was rather arbitrary.

7.3 Larger areas and multiple stations

Subsection 6.6.3 identified the interest of considering larger areas and potentially multiple stations. This section explains the problem in further details, then identifies which parts of the initial model need to be changed and finally presents a solution.

7.3.1 The problem

The initial model was tested on a rather small example (a subset of the station of Courtrai). It would be interesting to test the performance of CP on larger areas to see if the model is scalable in that way.

7.3.2 Parts of the model to be modified

As explained in Subsection 6.2.1, each train has one and only one itinerary activity. This is the main limitation in the case we want to avoid changing the structures of the layout we consider for the stations *i.e.* the routes and itineraries.

7.3.3 Solution

It is possible to implement this extension by adding the itineraries as parameters of the train. Table 7.3 shows the differences in parameters when adding the train length, platforms and the

Table 7.3: Parameters of trains and track segments after the addition of train length, platforms and consecutive itineraries

Entity	Parameter	Name	Meaning
Train	Speed	spd_t	Speed of t
	Passengers	p_t	Number of passengers of t
	Estimated Arrival Time	eat_t	When t will arrive at the station
	Earliest Start Time	est_t	Lower bound on the start time of t
	Planned Completion Time	pct_t	When t is supposed to complete its journey
	Category	c_t	Category of t
	Length	lgt_t	Length of t
	Stops	$stops_t$	List of platforms where t should stop at
	Minimum Stopping Times	$times_t$	List of minimum amount of time t should stop for every stop
	Journey	j_t	List of itineraries of train t
Track segment	Length	lgt_{ts}	Length of ts

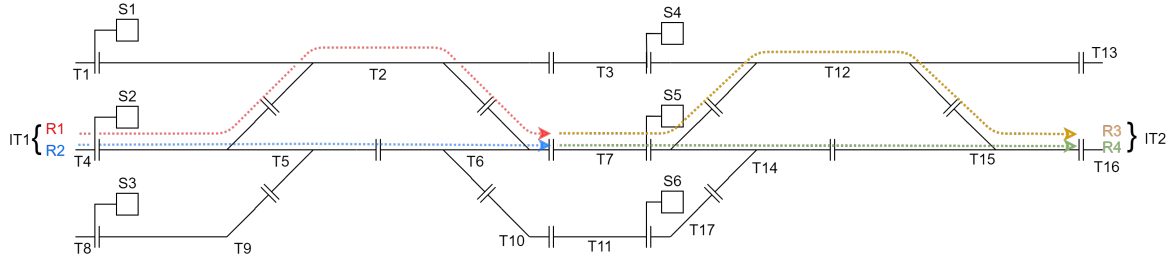


Figure 7.8: Example of a toy station with consecutive itineraries

possibility for a train to have multiple itineraries. The following changes are applied to the parameters:

- The multiple itineraries of a train t are treated as a list of itineraries that we call a **journey** j_t , where $j_t(0)$ is the first itinerary to complete, $j_t(1)$ the second one, etc. These itineraries should be consecutive. With N_t^j the size of j_t , $dest(j_t(i))$ the destination of the $(i + 1)^{th}$ itinerary of t and $orig(j_t(i))$ its origin, the consecutiveness of the itineraries is expressed by:

$$dest(j_t(i - 1)) = orig(j_t(i)) \quad \forall t \in T, \forall i \in [1, N_t^j] \quad (7.20)$$

Figure 7.8 shows an extension of the toy station presented in Figure 6.1 to illustrate the concept of consecutive itineraries. In this figure, $IT1$ and $IT2$ are considered as consecutive because $dest(IT1) = orig(IT2) = T7$.

- Because a train may stop for every itinerary, we need to specify for each of them whether it has to stop, where to do it and for what minimum time. Let us define $stops_t$ as a list of platforms where the train should stop for each itinerary and $times_t$ as a list of minimum

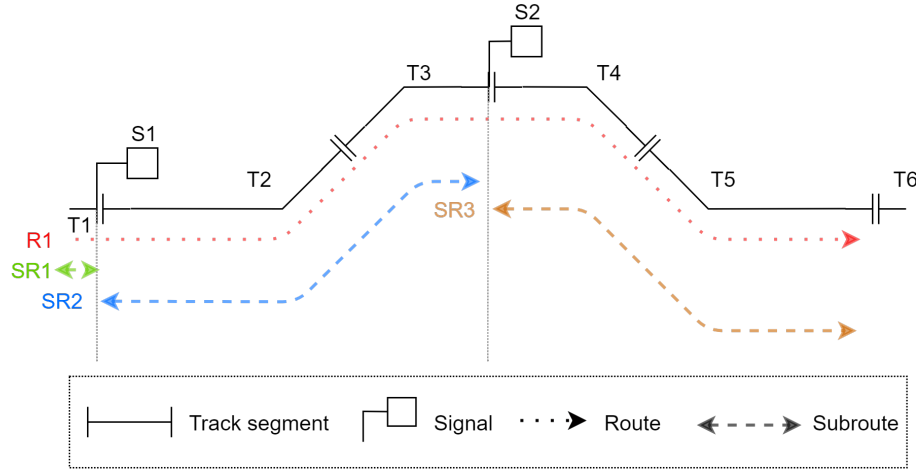


Figure 7.9: Example of a route split into multiple subroutes

stopping time for each platform. Concretely, $stops_t(0)$ specifies the platform where t should stop during its first itinerary *i.e.* the ssa_t of the first itinerary, $times_t(0)$ specifies for at least how long *i.e.* the mst_t of the first itinerary, $stops_t(1)$ and $times_t(1)$ concern the second itinerary of t , etc.

The following constraint is used to link the itineraries in the model:

$$s(A^{t,j_t(i)}) = e(A^{t,j_t(i-1)}) \quad \forall t \in T, \forall i \in [1, N_t^j] \quad (7.21)$$

This constraint states that the starting date of the $(i + 1)^{th}$ itinerary of the train should be the same as the end of the i^{th} itinerary. This means that when a train finishes an itinerary, it starts the subsequent one immediately.

7.4 Encountering signals after route activation

As seen in Subsection 6.6.4, it may happen in reality that a train stops after its route activation. Section 7.2 explained how to handle the fact of stopping at a platform, this also works in case of signals. However, with this possibility, the reservation of the resources is open to changes. There are two approaches that are considered.

7.4.1 Reservation of the whole route after activation

The first possibility is to reserve all of the elements of the route after the route activation. This is what was done in the initial model.

7.4.2 Reservation of the elements until a signal or a platform

The other possibility, the one used in the new model, is to define **subroutes** as parts of routes delimited by signals or platforms. A similar concept called *blocks* is presented in [20]. The idea is to reserve all track segments up until the next signal or platform encountered within

the route. Then proceed as if there was another route activation for the next subroute. An example of subroutes for a route is shown on Figure 7.9. On the figure, route $R1$ has been split on signals into three subroutes: $SR1$, $SR2$ and $SR3$. The elements of these structures are $[T1, S1, T2, T3, S2, T4, T5]$ for $R1$, and $[T1, S1]$, $[T2, T3, S2]$ and $[T4, T5]$ for the subroutes, respectively.

This approach is interesting as it gives more flexibility to the model. Indeed, there may possibly be a lot of track segments located after a signal that would not be reserved until this part of the route has been activated (which could take some time if the preceding subroute is long), leaving them free to use to other trains in the meanwhile. For instance, in Figure 7.9, the elements of $SR2$ and $SR3$ are free to use by other trains during the time the train is on $SR1$.

Let us recap the hierarchy of the logical structure after the addition of subroutes:

- A subroute is a path that goes from the origin of an itinerary, a signal or a platform until a signal, a platform or the end of an itinerary.
- A route is a path that a train can use in order to reach a destination from its origin. It can contain multiple subroutes.
- An itinerary is a path between two specific locations, generally within the same station or between stations. It is possible that an itinerary contains one or more alternative routes.
- A journey is a list of consecutive itineraries. It can span across multiple stations.

7.5 Conclusion about the new model

To wrap up, this section presented our new model and all the modifications it brings to the initial model presented in the last chapter.

Some of the advantages and drawbacks of our model compared to the original model are listed below.

7.5.1 Advantages

The first advantage of the new model is that it is closer to the reality and more general. Indeed, the addition of the concepts of length of trains, platforms, and encountering signals and platforms after route activation allow the model to consider more realistic cases as well as a wider range of possible stations.

The model also allows to consider large areas, with multiple stations if needed. This gives the possibility to study the performances of models which consider the infrastructure at a microscopic level on such kind of areas.

The last advantage is that our model may give better results than the initial model because of the reservation system presented in Subsection 7.4.2.

7.5.2 Drawbacks

The main drawback of our model is that it is a bit slower than the initial model because of the addition of extra decision variable and constraints. Because the search phase is slower, the new model may not give as good solutions as the initial one.

7.6 Further work

This section presents some additional features that might be interesting to implement in the model.

7.6.1 Considering the type of platforms

It may happen that some platforms are too short or not fitted for some trains. These constraints would be interesting to consider as well as when taking the decision to reroute the train

7.6.2 Variable speed of the trains

One more aspect that is not considered in the model is the fact that trains have dynamically changing speeds. Indeed, the parameter spd_t is supposed to be constant during the whole journey.

Multiple things could be taken into account here again:

- Some parts of the station might be under speed limitations. One could specify these areas on the station layout and add the corresponding constraints to enforce these rules.
- Because the trains are likely to stop and start at signals, it is unlikely that they keep the same speed all along their itineraries. An acceleration and deceleration factors could be specified for each train depending on other parameters *e.g.* its current speed, weight, etc.

This parameter was considered in the model of Rodriguez [20]. In this paper, the author states that the benefits of the addition of variable speed in the model are mitigated. Even though it gives a closer approximation of the reality, the computational costs underlying its integration to the model are significant. There is thus a trade-off between the performances and the accuracy of the model.

7.6.3 Considering train shunting

It may occur that multiple trains are assembled together at some point. This may for example take place when one of them is stopped at a platform. Multiple problems may arise in such cases.

- This situation leaves no flexibility when it comes to individual train routing. Indeed, the trains must be forced to stop at the same place until they are assembled.
- At some point the trains need to join each other and hence use the same resources. On the contrary to what is specified in Subsection 6.3.5, in these cases, it is acceptable for two activities of different trains to overlap.

7.6.4 Introduction of disturbances

Since the model simulates the actions of the components ahead, it would be interesting to find a way to introduce perturbations such as a faulty signal or an obstructed track segment as well in order to study how the model could react to those.

Chapter 8

Experiments

This chapter evaluates the performances of the CP model presented in the last chapter through different experiments. Practically, the results given by the CP model are compared with the one given by classical scheduling strategies, then the CP model is tested on critical situations and its scalability is evaluated.

All the tests are performed on the stations of Wavre and Ottignies (Appendix B). This layout contains 304 track segments, 69 signals, 13 platforms, 478 itineraries and 595 possible routes. The layout was directly extracted from files provided by the Belgian railways.

The cost of rerouting a train to another platform is always equal to 0 on the studied stations as they are encoded in a way that does not provide any flexibility to do such things (the platforms are located either as first or last element of the itineraries, where no rerouting through other platforms are possible). Even though it was tested on toy examples on our part, further work on the extraction of stations needs to be done in order to test this.

Three greedy approaches simulated via discrete event simulation are compared against CP:

- **First Come First Served (FCFS)**: this strategy gives the priority to the trains according to their arrival time.
- **Highest Delay First Served (HDFS)**: this strategy gives priority to the trains according to their planned completion time.
- **Highest Priority First Served (HPFS)**: this strategy gives priority to the trains according to their category, as explained in 6.4.2.

Three meta-parameters are considered during the experiments:

1. The *time horizon* defines an upper bound on the estimated arrival time of the trains. Three time horizons are considered in the experiments: 30, 60 and 120 minutes.
2. The *number of trains* gives the number of trains which will be scheduled to arrive at the stations during the time horizon specified. We experimented our model with 5, 10, 15, 20, 25 and 30 trains.
3. The *decision time* is the number of seconds the railway operator has at disposal to take a decision. Although it is dependent on the situation, Rodriguez [20] suggests a time limit of 3 minutes to be used in reasonable conditions.

Experiments have been performed with the assumption that only feasible schedules are provided to the model. That is, when both the model and the discrete event simulator built to simulate the classical strategies finds a solution. Heterogeneous and homogeneous random schedules were generated.

The experiments have been realised on a Dell XPS 13 9360 with a 2.5GHz Intel Core i5 processor (4 CPUs) and 8Gb of RAM at 1866 MHz using a 64 Bits OpenJDK Server VM 1.8 (by JetBrains) on Windows 10 Family edition V1803. The model was implemented with the Java API of CP Optimizer V12.7.1 and the search was performed with four workers.

8.1 Homogeneous traffic

This section assesses the performances of the model presented in Chapter 7 when considering homogeneous traffic. The number of passengers and categories of the trains are not considered here. Hence, Equation 7.18 is used as objective function.

Each configuration of meta-parameters is repeated 100 times with a random schedule and 3 minutes of decision time. The various values are generated with a uniform distribution. For example, for a schedule with 10 trains and an horizon of 120 minutes starting at 2pm, the program randomly chooses multiple consecutive itineraries *i.e.* a random journey and if possible a platform to stop at per itinerary with a random stopping time. A random length of the train in the interval $[0, 120)^1$ and a random speed in $[35, 47)$ units of distance per minute are also chosen. The estimated arrival time and departure time are generated randomly in the interval $[2pm, 4pm]$. The planned completion time is set to the sum of the estimated arrival time and the number of itineraries times a constant factor c *i.e.* $pct_t = eat_t + N_t^j \times c$.

Table 8.1 shows a part of the results of these experiments. The delays indicated in the first table are the arithmetic means over all the measures per configuration: each line contains the average delay of each configuration. POS indicates the number of times CP provides an equal or better solution than both other strategies while OPT indicates the number of times CP finds an optimal result.

As in [6], it appears that CP gives better solutions almost all the time (more than 99% of the cases). The model also gives optimum solution oftenly (more than 75% of the cases) for instances with 10 or less trains. Naturally, the average delay grows with the number of trains and decreases when enlarging the horizon.

Table 8.2 concerns the other part of these experiments. It gives the improvement ratio when comparing our CP model with the classical approaches: the average, best and worst improvement of CP against FCFS and of CP against HDFS are reported in each line.

In this table, the average improvement ratio is above 40% in all the scenarios except one. The best improvement ratio reaches sometimes 100%, that is when CP gives a solution with

¹The units of distance are relative to the layout of the station. The layout and data related to the train station used in the experiments can be found here: <https://github.com/ffelten/opendata>

Table 8.1: Results of CP and classical approaches for an homogeneous traffic and 3 minutes of decision time: delays, number of improvements and optimal solutions

Horizon	# trains	Average delay (min.)			POS (x/100)	OPT (x/100)
		FCFS	HDFS	CP		
30 minutes	30	1304.0	1303.0	863.12	100	0
	25	900.0	906.0	562.59	100	0
	20	660.0	665.0	411.81	100	0
	15	348.0	347.0	215.72	100	20
	10	143.0	146.0	108.59	100	62
	5	67.86	66.72	38.32	100	99
1 hour	30	1271.2	1277.35	735.87	100	0
	25	871.57	879.69	474.31	100	0
	20	616.71	623.65	340.69	100	2
	15	355.35	357.14	199.21	100	18
	10	181.93	182.76	99.15	100	70
	5	65.47	65.88	40.31	100	99
2 hours	30	1021.94	1027.49	527.19	99	0
	25	809.17	803.81	424.48	100	1
	20	485.09	484.22	246.67	100	12
	15	312.7	314.82	158.97	100	43
	10	161.35	163.21	87.5	100	81
	5	61.52	61.73	38.65	100	100

Table 8.2: Results of CP and classical approaches for an homogeneous traffic and 3 minutes of decision time: improvement ratios

Horizon	# trains	Avg. Improvement Ratio (%)		Best Improvement Ratio (%)		Worst Improvement Ratio (%)	
		CP vs FCFS	CP vs HDFS	CP vs FCFS	CP vs HDFS	CP vs FCFS	CP vs HDFS
30 minutes	30	38.39	37.88	62.83	57.92	1.16	2.27
	25	41.17	42.43	67.43	70.68	11.68	11.41
	20	43.01	43.68	70.02	67.37	10.8	19.29
	15	48.08	48.44	76.38	74.25	13.18	14.92
	10	45.46	45.81	80.52	80.52	9.77	9.77
	5	47.96	46.82	100.0	100.0	0.0	0.0
1 hour	30	43.41	43.73	69.35	67.94	11.36	11.36
	25	46.77	47.21	71.12	71.12	16.37	16.37
	20	46.26	46.9	69.35	71.54	16.43	15.71
	15	45.77	45.87	74.66	74.66	15.44	15.44
	10	46.54	46.76	100.0	100.0	16.18	16.18
	5	41.35	41.5	100.0	100.0	0.0	0.0
2 hours	30	49.56	49.73	77.5	76.89	-2.17	-2.17
	25	49.48	49.08	82.25	82.75	14.56	14.56
	20	49.76	49.73	70.0	70.0	11.31	11.75
	15	50.36	50.47	76.72	76.72	17.87	17.87
	10	45.12	45.41	90.91	90.91	14.29	14.29
	5	41.79	41.95	100.0	100.0	0.0	0.0

Table 8.3: Results of CP and classical approach (HPFS) for an heterogeneous traffic and 3 minutes of decision time

Horizon	Trains	POS	OPT
30 minutes	30	99	0
	25	100	0
	20	100	3
	15	99	29
	10	100	66
	5	98	96
1 hour	30	98	0
	25	98	1
	20	100	7
	15	99	36
	10	98	74
	5	99	99
2 hours	30	100	1
	25	99	7
	20	100	28
	15	100	56
	10	99	73
	5	100	99

no delay while the other strategy gives a solution with delay. On the other hand, the worst improvement ratio has a value of 0 in every scenario implying 5 trains, that is when CP gives the same solution as the greedy approach.

8.2 Heterogeneous traffic

This section studies the performances of the model on an heterogeneous traffic. Equation 7.19 is used as objective function. The number of passengers and categories of trains are then considered in these experiments.

The scenarios are generated randomly as in the previous section for all the common parts. The category of the train is chosen in a random fashion according to a uniform distribution among the four possibilities. The number of passenger is chosen the same way in the interval $[0, 200)$, for passenger and correspondence trains.

As explained in [6], only HPFS deals with an heterogeneous traffic. The optimisations are performed one after the other for each part of the objective function as advised in [1]. The search time is allocated according to the priority of the categories with an upper bound of 60% of the remaining time at each step (except the last one which uses all the remaining time).

Table 8.3 reports the results of the experiments. Unlike in the previous section, the delays and improvement ratios are not reported in this table. This is motivated by the fact that it is subjective to define an improvement ratio in such traffic. Indeed, it is up to the railway operator

Table 8.4: Comparison between a critical CP model (10 seconds of decision time) and a normal CP model (3 minutes of decision time) on an homogeneous traffic

Horizon	# Trains	Avg. Delay crit. CP	Avg. Delay CP	Avg. Improvement Ratio (%)	Best Improvement Ratio (%)	OPT _{crit}	OPT
30 minutes	5	51.6	51.01	0.92	42.5	83	99
	10	122.36	116.0	4.23	50.0	15	62
	15	280.07	247.82	8.92	52.93	5	12
	20	601.39	445.92	23.0	54.37	0	1
	25	933.9	669.29	26.18	57.58	0	0
	30	1398.9	987.1	27.34	68.41	0	0
1 hour	5	36.67	36.59	0.07	5.49	89	99
	10	99.53	97.35	1.77	27.03	41	69
	15	233.96	203.31	8.56	54.72	4	23
	20	462.07	364.15	17.78	64.71	1	2
	25	732.74	539.92	22.73	54.77	0	0
	30	1128.28	831.08	25.36	56.78	0	0
2 hours	5	41.41	41.29	0.17	17.39	93	100
	10	90.07	88.56	1.39	36.84	56	81
	15	177.35	163.23	4.87	36.91	13	38
	20	311.7	260.33	12.33	55.94	2	9
	25	535.43	401.76	20.68	58.96	0	0
	30	703.73	546.64	19.69	48.71	0	0

to decide whether the reduction of the delay of one train of category C1 against 5 trains of a lower priority is worth it. As in [6], we consider that CP gives a better solution than HPFS if its delay per category is lexicographically lower than the solution provided by the greedy approach.

In this kind of traffic, the CP model gives a better or equal solution in almost all the cases as well (around 99% of the cases). It also gives optimum solution in more than 75% of the cases when considering 10 trains or less.

8.3 Critical situations

This section studies the variation of performances of the model when facing critical situations *i.e.* with a small decision time. Concretely, the scenarios are generated the same way as with the homogeneous traffic. Two configurations of the model are launched on every scenario: one with 10 seconds of decision time (the critical CP model) and the other with 3 minutes (the uncritical model). Only scenarios where the critical CP gives a solution are considered.

Table 8.4 recaps the results of the experiments where the critical model found a solution. As in Section 8.1, the average delay is the arithmetic mean of the delays over the 100 runs per configuration of meta-parameters. The improvement ratios quantifies by how much the uncritical CP model gives better solutions than the critical one. OPT indicates the number of times the non critical model provides an optimal solution whereas OPT_{crit} gives the same figure for the critical one. The worst improvement ratio does not appear as it is always equal to zero.

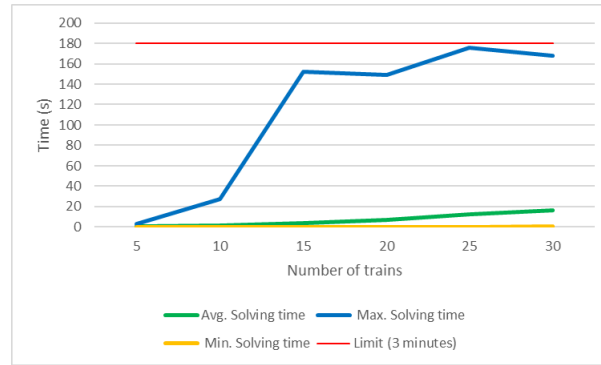


Figure 8.1: Evolution of the computing time needed to obtain a first solution over 100 random homogeneous instances with an horizon of 30 minutes

It appears that the model overall behaves well with little decision time. On small instances (15 or less trains), the average improvement ratio is always less than 10%. On bigger instances, it may be worthy to give more decision time if available as the average improvement is always over 10% in those cases. Obviously, the number of optimal solutions increases when the model benefits of more decision time.

By comparing the average improvement ratios in Tables 8.2 and 8.4, it appears that in general the critical CP model gives better results than classical heuristics.

8.4 Scability

It has been showed in [6] that there is no real gain in increasing the decision time to more than 3 minutes for the initial model. The previous sections shown the model to be efficient even on multiple stations. In this section, we are interested in the minimum amount of time the model needs to find a solution.

Figure 8.1 gives the evolution of the computing time needed to have a solution for an instance over 100 random homogeneous scenarios as a function of the size of the trains. The average, maximum and minimum time over all the tests are reported. The horizon is set to 30 minutes as it generates the most constrained scenarios (because of the density of trains within a short time window). Only the cases where a solution is found within 3 minutes are considered.

The model performs well in average in terms of scalability: the average solving time needed to obtain a first solution is less than 20 seconds for up to 30 trains. However, some cases occur where the model needs up to 3 minutes time to find a solution. This leads us to think that some instances may lead to timeouts. Some further experiments with real data would be interesting to consider to assess this.

8.5 Reproducibility

Files and information to reproduce our results are available on <https://github.com/ffelten/opensdata>. Using this document and the files provided on GitHub, it is possible to build a model and perform the same experiments as we did.

Chapter 9

Conclusion

This work aimed at reaching two objectives, which correspond to the two parts this document is composed of.

The first was to propose an implementation of conditional time-intervals for OscanR, and evaluate if they brought advantages compared to traditional scheduling modeling techniques. The implementation is complete and functional. Experiments on the networks regarding their structures and propagation algorithms might be interesting in order to speed the process but, while some further work is still needed to reach the performance achieved in CP Optimizer, the new variables are already more powerful than traditional variables in a range of problems. They also benefit from ease of use and the fact that both networks are completely invisible to the end user. Along with the constraints that were implemented for them, they allow for concise models that cover a wide variety of scheduling problems.

The second objective was to propose additions to the model of Cappart and Schaus [6], and to benchmark their performances. The model presented in this document handles trains of variable lengths, stops at platforms, more general layout of stations, and a way to manage large areas is proposed. It has been tested on fictive data and compared to classical approaches (HDFS, FCFS, HPFS). Even with the addition of these extensions, the model outperforms the greedy approaches in almost all the cases. It is suitable to be used in real-time applications and scalable in most cases. As a matter of fact, it is able to handle scenarios with up to 30 trains in less than 20 seconds in average, yet some cases may lead to timeout. Further work with real data is required to assess this aspect.

There are many opportunities to extend this work. Experiments with real data can be conducted to assess the performances of the model in real conditions. It is also possible to extend the model to have a closer simulation of the reality, for instance by considering trains with dynamically changing speed or train shunting. Finally, the introduction of unexpected events (obstructed tracks, faulty signal, etc.) could be interesting to consider.

Bibliography

- [1] CP Optimizer User's Manual Version 12, Release 7, 2017.
- [2] Roman Barták, Miguel A. Salido, and Francesca Rossi. New trends in constraint satisfaction, planning, and scheduling: a survey. *The Knowledge Engineering Review*, 25(3):249–279, September 2010.
- [3] Ronen I. Brafman. A simplifier for propositional formulas with many binary clauses. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 34(1):52–59, February 2004.
- [4] Valentina Cacchiani, Dennis Huisman, Martin Kidd, Leo Kroon, Paolo Toth, Lucas Vee-
lenturf, and Joris Wagenaar. An overview of recovery models and algorithms for real-time
railway rescheduling. *Transportation Research Part B: Methodological*, 63:15–37, May 2014.
- [5] Quentin Cappart. *Verification of railway interlocking systems and optimisation of railway
traffic*. PhD thesis, UCL - Université Catholique de Louvain, UCL - Université Catholique
de Louvain, 2017.
- [6] Quentin Cappart and Pierre Schaus. Rescheduling Railway Traffic on Real Time Situations
Using Time-Interval Variables. In *Integration of AI and OR Techniques in Constraint
Programming*, Lecture Notes in Computer Science, pages 312–327. Springer, Cham, June
2017.
- [7] Amadeo Cesta and Angelo Oddi. Gaining efficiency and flexibility in the simple temporal
problem. *Proceedings Third International Workshop on Temporal Representation and
Reasoning*, 1996.
- [8] Andrea D'Ariano, Francesco Corman, Dario Pacciarelli, and Marco Pranzo. Reordering and
Local Rerouting Strategies to Manage Train Traffic in Real Time. *Transportation Science*,
42(4):405–419, November 2008.
- [9] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. A branch and bound algorithm
for scheduling trains in a railway network. *European Journal of Operational Research*,
183(2):643–657, December 2007.
- [10] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. Assessment of flexible timetables
in real-time traffic management of a railway bottleneck. *Transportation Research Part C:
Emerging Technologies*, 16(2):232–245, April 2008.

- [11] Steven Gay, Renaud Hartert, Christophe Lecoutre, and Pierre Schaus. Conflict Ordering Search for Scheduling Problems. In Gilles Pesant, editor, *Principles and Practice of Constraint Programming*, volume 9255, pages 140–148. Springer International Publishing, Cham, 2015.
- [12] Robert M. Haralick and Gordon L. Elliott. Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313, October 1980.
- [13] IBM. CPLEX CP Optimizer, 2009. Available from <https://www.ibm.com/analytics/data-science/prescriptive-analytics/cplex-cp-optimizer>.
- [14] Philippe Laborie and Jerome Rogerie. Reasoning with Conditional Time-Intervals. In *Proceedings of the 21th International Florida Artificial Intelligence Research Society Conference, FLAIRS-21*, pages 555–560, January 2008.
- [15] Philippe Laborie, Jerome Rogerie, Paul Shaw, and Petr Vilím. Reasoning with Conditional Time-Intervals. Part II: An Algebraical Model for Resources. January 2009.
- [16] Christophe Lecoutre, Lakhdar Saïs, Sébastien Tabary, and Vincent Vidal. Reasoning from last conflict(s) in constraint programming. *Artificial Intelligence*, 173(18):1592–1614, December 2009.
- [17] Alessandro Mascis and Dario Pacciarelli. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research*, 143(3):498–517, December 2002.
- [18] OsaR Team. OsaR: Scala in OR. 2012. Available from <https://bitbucket.org/oscarlib/oscar>.
- [19] OsaR Team. OsaR: Scala in OR. *Version with CTI variables*. 2018. Available from <https://bitbucket.org/jerthiry/oscar-cti>.
- [20] Joaquín Rodríguez. A constraint programming model for real-time train scheduling at junctions. *Transportation Research Part B: Methodological*, 41(2):231–245, February 2007.
- [21] Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

Appendix A

CPCTIVar API

This appendix introduces the CPCTIVar API in OcaR. Only the main methods are presented here. The constraints can be instantiated as explained in Section 5.4.

class **CPCTIVar** extends **CPVar**

Instance constructors

```
new CPCTIVar(store: core.CPStore, s: CPTimePointVar, e: CPTimePointVar, d: CPTimePointVar, exec: core.variables.CPBoolVar, _name: String)
```

Value members

```
def <==>(a: CPCTIVar): core.Constraint  
    Create an Equivalence constraint  
  
a          the other participant of the equivalence  
returns    an Equivalence(Array(this, a))
```

```
def =>(a: CPCTIVar): core.Constraint  
    Create an Implication constraint  
  
a          the consequent of the implication  
returns    an Implication(this, a)
```

```
def apply(timePoint: TimePoint): core.variables.CPIntVar  
    Getter for the timepoint of the interval variable  
  
timePoint  Timepoint.START or Timepoint.END  
returns    The CPIntVar for the start if START, for the end if END
```

```
def assignDuration(value: Int): Unit  
    Assign a value as a duration value  
  
value      The value to assign  
  
Exceptions thrown  Inconsistency when the value is not a possible value
```

```
def assignEnd(value: Int): Unit
```

Assign a value as an end value

value The value to assign

Exceptions thrown **Inconsistency** when the value is not a possible value

```
def assignFalse(): Unit
```

State that the interval will not be executed

```
def assignStart(value: Int): Unit
```

Assign a value as a start value

value The value to assign

Exceptions thrown **Inconsistency** when the value is not a possible value

```
def assignTrue(): Unit
```

State that the interval has to be executed

```
def implies(a: CPCTIVar): Boolean
```

Checks whether this variable implies a

a CPCTIVar

returns true if this implies the execution of a, false otherwise

```
def isFalse: Boolean
```

Checks if the interval will not to be executed

returns true if the execution status is fixed to false (0)

```
def isTrue: Boolean
```

Checks if the interval has to be executed

returns true if the execution status is fixed to true (1)

```
def name: String
```

Gets the name of the variable

returns the name of the variable

```
def setName(n: String): Unit
```

Sets the name of the variable

n the name to set

```
def updateMax(i: Int, t: TimePoint): Unit
```

Updates the maximum value for the timepoint t to i

i the new maximum value

t the timepoint (START or END)

```
def updateMin(i: Int, t: TimePoint): Unit
```

Updates the minimum value for the timepoint t to i

i the new minimum value
t the timepoint (START or END)

companion object **CPCTIVar**

Value members

```
def apply(name: String)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

name the name of the variable
store the CP store
returns A new CPCTIVar with every possible value

```
def apply(d: Int, name: String)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

d the value for the duration of the interval
name the name of the variable
store the CP store
returns A new CPCTIVar with every possible duration value

```
def apply(d: Int)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

d the value for the duration of the interval
store the CP store
returns A new CPCTIVar with every possible duration value

```
def apply(d: Array[Int], name: String)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

d the possible values for the duration of the interval
name the name of the variable
store the CP store
returns A new CPCTIVar with every possible duration value

```
def apply(d: Array[Int])(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

d the possible values for the duration of the interval
store the CP store
returns A new CPCTIVar with every possible duration value

```
def apply(d: Range, name: String)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

name the name of the variable
store the CP store
returns A new CPCTIVar

```
def apply(d: Range)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

store the CP store
returns A new CPCTIVar

```
def apply(s: Range, e: Range, name: String)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
name the name of the variable
store the CP store
returns A new CPCTIVar

```
def apply(s: Range, e: Range)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
store the CP store
returns A new CPCTIVar

```
def apply(s: Array[Int], e: Array[Int], name: String)(implicit store: core.CPStore):  

CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
name the name of the variable
store the CP store
returns A new CPCTIVar with every possible duration value

```
def apply(s: Array[Int], e: Array[Int])(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
store the CP store
returns A new CPCTIVar with every possible duration value

```
def apply(s: Range, e: Range, d: Range, name: String)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
d the possible values for the duration of the interval
name the name of the variable
store the CP store
returns A new CPCTIVar

```
def apply(s: Range, e: Range, d: Range)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
d the possible values for the duration of the interval
store the CP store
returns A new CPCTIVar

```
def apply(s: Array[Int], e: Array[Int], d: Array[Int], name: String)(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
d the possible values for the duration of the interval
name the name of the variable
store the CP store
returns A new CPCTIVar

```
def apply(s: Array[Int], e: Array[Int], d: Array[Int])(implicit store: core.CPStore): CPCTIVar
```

Create a new CPCTIVar

s the possible values for the start of the interval
e the possible values for the end of the interval
d the possible values for the duration of the interval
store the CP store
returns A new CPCTIVar

```
def maxValue(): Int
```

Get the max value for the components of the interval

returns the max value for the components of the interval (default: 10000)

```
def setMaxValue(value: Int): Unit
```

Updates the default max value (10000)

value the new default max value

Appendix B

Ottignies and Wavre

A miniature of the layout of the train stations of Wavre and Ottignies is exposed in Figure B.1. A bigger version of the image is available on https://github.com/ffelten/opendata/blob/master/ottignies-wavre/ottignies-wavre_layout.pdf.

The station of Ottignies is located on the direct line between Namur and Brussels in Wallonia. The station of Wavre is a smaller local station.

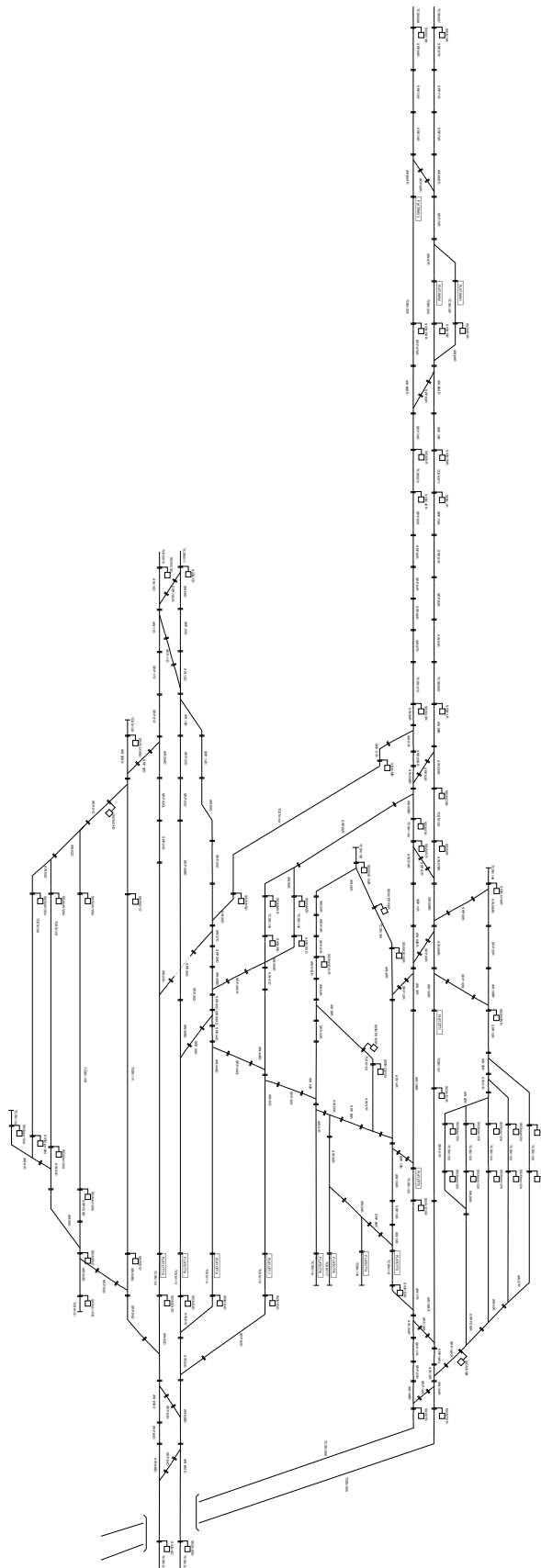


Figure B.1: The train stations of Wavre and Ottignies

Appendix C

Performances comparison: CP Optimizer and OsaR

The models presented in Chapter 7 are implemented with both CP Optimizer and OsaR. Their performances are compared in Tables C.1 and C.2. As CP Optimizer is multi-threaded and OsaR is not, the number of threads for CP Optimizer is limited to 1 for this test, where 3 minutes are allowed to optimize the delays of 5, 10, 15, 20, 25 and 30 trains, both homogeneous and heterogeneous¹ traffic. The results from the OsaR models were achieved with a Large Neighbourhood Search, and are the best out of different relaxation settings. CP Optimizer outperforms OsaR in every test, which is why the performances assessment compared to traditional methods (*e.g.* HDFS or FCFS) was done using the models implemented with CP Optimizer.

Table C.1: Comparison of the performances of CP Optimizer and OsaR (homogeneous model)

	CP Optimizer		OsaR	
# Trains	Time (s)	Objective	Time (s)	Objective
5	6,39	394	180	394
10	180	837	180	1011
15	180	1247	180	1729
20	180	2887	180	3729
25	180	4659	180	<i>timeout</i>
30	180	7100	180	<i>timeout</i>

Table C.2: Comparison of the performances of CP Optimizer and OsaR (heterogeneous model)

	CP Optimizer		OsaR	
# Trains	Time (s)	Objective	Time (s)	Objective
5	0,09	383	180	383
10	180	1174	180	1400
15	180	1932	180	2006
20	180	4228	180	4622
25	180	6634	180	7529
30	180	12890	180	<i>timeout</i>

¹For simplicity, the different objectives of the heterogeneous model were aggregated into one single objective in the table. This has no influence on the conclusion as every single objective was outperformed by the CP Optimizer model.

Appendix D

OscAR models for performance assessment

In this appendix, the models used to assess search performances and performances against models using reified constraints in OscAR are presented. The search method of each model is not presented as it was different at each run.

Algorithm 7 Model 1 (COP)

```
private val cp = new CPSolver()
private val activities = Array.fill(nVars)(CPCTIVar(
    random.nextInt(20) to random.nextInt(100) + 400,
    random.nextInt(500) + 30 to random.nextInt(1000) + 600)(cp))

for (i <- activities.indices) {
    activities(i).setName(s"ACT_$_i")
    activities(i)(DURATION).updateMin(random.nextInt(3))
}
var i = 1
while (i < activities.length) {
    cp.add(Equivalence(activities.slice(i + 1, i + 10)))
    cp.add(Span(activities(i), activities.slice(i + 1, i + 10)))
    cp.add(NoOverlap(activities.slice(i + 1, i + 10)))
    i += 10
}
cp.add(Alternative(activities(0),
    activities.indices.filter(_ % 10 == 1).map(activities(_))))
activities(0).assignTrue()

cp.minimize(activities(0)(END))
```

Algorithm 8 Model 2 (CSP)

```

private val cp = new CPSolver()

private val activities = ArrayBuffer.empty[CPCTIVar]
activities += CPCTIVar(s"ACT_0")(cp)

for(i <- 1 to nVars) {
    activities += CPCTIVar(
        random.nextInt(20) to random.nextInt(100) + 1000,
        random.nextInt(500) + 30 to random.nextInt(1000) + 1100)(cp)
    activities(i).setName(s"ACT_{$i}")
    activities(i).assignTrue()
}

cp.add(Span(activities(0), activities.slice(1, nVars)))
cp.add(NoOverlap(activities.slice(1, nVars)))

cp.minimize(activities(0)(END))

```

Algorithm 9 Model 3 (COP)

```

private val cp = new CPSolver()
private val activities = ArrayBuffer.empty[CPCTIVar]
for(i <- 0 to nVars) {
    activities += CPCTIVar(s"ACT_{$i}")(cp)
    activities(i)(DURATION).updateMin(random.nextInt(25))
}

private val obj = sum(activities.map(cti => cti(EXEC)*cti(DURATION)))

private val act = CTIArray2IntArray(activities)

private val count = sum(activities.map(cti => cti(EXEC)))
cp.add(count == nVars/4)

cp.minimize(obj)

```

Algorithm 10 Model 4 (COP)

```

private val cp = new CPSolver()
private val activities = ArrayBuffer.empty[CPCTIVar]
for (i <- 0 until nVars) {
    activities += CPCTIVar(
        random.nextInt(20) to random.nextInt(100) + 400,
        random.nextInt(500) + 30 to random.nextInt(1000) + 600)(cp)
}

for (_ <- 0 until nVars/2) {
    val i = random.nextInt(nVars)
    val j = random.nextInt(nVars)
    cp.add(activities(i) ==> activities(j))
}

for (i <- 0 until nVars by 10) {
    cp.add(Span(activities(i), activities.slice(i+1, i+10)))
    cp.add(Equivalence(activities.slice(i+1, i+10).toArray))
}

private val s = sum(activities.indices.map(i =>
    activities(i)(EXEC) * activities(i)(END)))
cp.add(s <= 300 * nVars)
private val count = sum(activities.map(cti => cti(EXEC)))

cp.maximize(count)

```
